UNIVERSITY OF WEST ATTICA
FACULTY OF ENGINEERING
DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING

Thesis:

Design and Implementation of Soulbound Tokens (SBTs) with Zero-Knowledge Proofs (ZKPs) for Decentralized Identity and Privacy

Writer: Iris Vereniki Eleanna Cacoulidi

Student ID: 21387195

Supervisor: Dimitrios Kogias

ATHENS-EGALEO, JUNE 2026

Chapter 1-Introduction

1.1 The Digital Identity Problem

As competition in the job market continues to rise, individuals increasingly rely on a combination of professional credentials, certifications, project-based experience, internships, and additional skills to demonstrate their qualifications. However, proving the legitimacy and scope of these qualifications is difficult within a fragmented digital environment.

For example, an individual applying for a software engineering role may possess a degree from an accredited institution, internship experience, personal projects, and endorsements from colleagues or supervisors. However, presenting these qualifications to a prospective employer often requires significant time and effort spent researching, navigating multiple platforms, and manually compiling credentials. At the same time, employers cannot easily verify the authenticity of these qualifications without directly contacting each issuing institution. Consequently, the process becomes inefficient, slow, fragmented, and heavily dependent on institutional intermediaries, leaving both parties in a position of uncertainty.

In fact, this scenario is not just an edge case but an actual everyday experience for professionals around the world. The method of managing a professional identity in the digital world has three critical and mutually reinforcing structural failures; fragmentation, fraud vulnerability, and the absence of meaningful user control.

Fragmentation

Professional credentials exist in incompatible systems governed by separate institutions such as the university, employer, certification body, and professional networks. There is no universal standard for representing, sharing, or verifying a credential across these institutional boundaries. For instance, a W3C Verifiable Credential issued by a university in Europe cannot be directly used at an employment platform built on a different data model (W3C, 2019, §5.3,§5.4). The same issue exists with a LinkedIn endorsement which carries no cryptographic proof of authenticity. And as for a PDF degree certificate it can be easily reproduced with freely available tools. The cumulative effect is that a professional identity must be reassembled from scratch for each new context, resulting in unnecessary friction and verification efforts for both credential holders and credential verifiers (Allen, 2016).

Fraud

Credential fraud is a significant and growing issue across sectors globally. The Association of Certified Fraud Examiners estimates that organisations globally collectively lose 5% of revenue each year, nearly $5 trillion due to occupational fraud, including some aspect of false or misrepresented credentials (ACFE, 2024). Degree mills generate over a billion dollars in revenue each year from providing fraudulent degrees and other forms of academic credential (Ezell & Bear, 2012). Traditional methods of credential verification require contacting the issuing institutions, which is highly labour-intensive, slow, and requires significant resources and time to complete. Automated pre-employment screening services are partially effective at solving this problem, as they use proprietary databases that are limited in scope and not globally interoperable.

Lack of User Control

Even when credentials are verified through an external source, users continue to have very little direct influence over the way their identity information is managed and shared. Professional platform providers such as LinkedIn retain and monetise user profile data. University registry systems store records indefinitely, typically with no mechanism by which graduates can limit or revoke third-party access. Employment background-screening services aggregate and resell sensitive personal information with limited consent infrastructure (GDPR, 2016, Recital 6, Recital 32) The individual is simultaneously the primary subject of their own credentials and the least empowered actor in the verification ecosystem.

These three structural failures together constitute what this thesis tries to address the digital identity crisis: a systemic inadequacy of existing infrastructure to deliver portable, cryptographically verifiable, and user-controlled professional credentials.

1.2 Why Blockchain-Native Identity Matters Now

The structural failures identified above share a common source, they arise from the dependence of credential systems on centralised, siloed databases controlled by individual institutions. In contrast, distributed ledger technology offers a fundamentally different architecture one in which records are maintained collectively, tamper-evidence is enforced cryptographically, and no single institution controls the authoritative copy. Three characteristics of this technology are of importance to professional identity systems.

Immutability and Tamper-Evidence

Once a record is committed to a blockchain, altering it requires controlling most of the network's consensus participants without this property, credential forgery will be almost computationally impossible, at least for larger applications. When an employer is accessing a credential stored on-chain they receive a cryptographic assurance of authenticity that neither a PDF certificate nor a centralised database lookup can provide (Nakamoto, 2008; Wood, 2014).

Decentralisation

Unlike centralised identity providers national e-ID systems, social login services, or institutional registries blockchain-based identity is not owned or controlled by any single organisation. Users interact directly with deployed smart contracts; as such, there is no intermediary capable of unilaterally revoking access, censoring records, or becoming a single point of failure, satisfying the fundamental requirement that identity must not be held by a singular third-party entity (Allen, 2016; Wood, 2014). This architecture is especially valuable in cross-border professional mobility contexts, where no single jurisdiction's identity infrastructure commands universal trust or interoperability.

Programmability

Smart contracts allow credential and identity logic to be encoded as executable, self-enforcing rules deployed directly on the blockchain. Access control policies, credential issuance conditions, and verification requirements can all be expressed in code that executes identically for every participant, without reliance on trusted intermediaries (Szabo, 1997; Wood, 2014). This programmability transforms the blockchain from a passive record store into an active enforcement layer for identity governance.

The maturity of blockchain infrastructure has reached a point at which these properties can be exploited for practical, production-oriented identity systems. The Ethereum Virtual Machine (EVM) and its compatible chains support a rich ecosystem of tooling, standards, and developer tooling. Enterprise-grade consortium blockchains such as Hyperledger Besu extend this programmability into permissioned settings with deterministic finality and predictable transaction costs (Hyperledger Besu, n.d.) characteristics well-suited to institutional identity deployments.

The World Wide Web Consortium (W3C) has formally recognised the potential of this architecture through its Decentralised Identifiers (DID) specification (W3C, 2022) and Verifiable Credentials (VC) data model (W3C, 2019), which establish a standards layer compatible with blockchain infrastructure. What remains absent, however, is a concrete integrated system combining these standards with mechanisms for non-transferability, granular access control, and privacy-preserving verification. This thesis addresses that gap.

1.3 Soulbound Tokens as a Paradigm Shift

In January 2022, Vitalik Buterin introduced the concept of 'soulbound' tokens blockchain tokens that are permanently and non-transferably bound to a single wallet address, drawing an analogy to the 'soulbound' items in certain role-playing video games that become permanently bound to a character upon acquisition and cannot subsequently be traded (Buterin, 2022a). The implications for identity systems are consequential: if a credential is represented as a token that cannot be transferred, sold, or delegated, then possession of that token constitutes evidence of legitimate ownership an assurance that freely transferable token standards such as ERC-20 or ERC-721 cannot provide.

Buterin, Weyl, and Ohlhaver subsequently formalised this concept in Decentralized Society: Finding Web3's Soul (Weyl, Ohlhaver & Buterin, 2022), which articulated a broader vision of a 'Decentralized Society' (DeSoc) constructed on a network of non-transferable attestations educational credentials, employment records, and community memberships stored as Soulbound Tokens (SBTs) in user-controlled 'Soul' wallets. The authors argued that SBTs could serve as a foundation for a pluralistic, bottom-up model of social trust that complements, rather than replaces, existing institutional structures.

The ERC-5484 standard, proposed in 2022 and ratified in 2023, provides a concrete EVM implementation of non-transferable tokens (Cai, 2022). It extends the ERC-721 non-fungible token standard with a burnAuth parameter that encodes which party holds the authority to destroy a given token: the issuing institution, the token holder, both parties jointly, or neither. This fine-grained burn control enables diverse deployment scenarios a university issuing a degree credential might reserve burn authority exclusively to itself, while a self-asserted skill badge might be burnable by the holder alone without requiring separate contract implementations for each case.

SBTs represent a conceptual reorientation of the blockchain token: from a tradeable financial instrument to a verifiable social attestation. This shift creates the foundation for a genuinely portable and interoperable professional identity, one that travels with the individual rather than residing in any single platform's proprietary database.

1.4 The Privacy Paradox on Public Blockchains

The adoption of SBTs for credential management introduces a fundamental tension not encountered in conventional identity systems. The transparency that makes public blockchains effective gives it the ability to make the record immune from tampering. However, it is precisely the same transparency that creates challenges for storing sensitive personal data in its raw form.

All data committed to a public blockchain can be permanently accessed by anyone in the network (or anyone who is able to see the public blockchain through a block explorer). This includes all transaction information, all contract invocation information, and all value information. For financial assets, this transparency functions as an accountability mechanism. For professional credentials, it constitutes a significant liability. A job applicant should be able to demonstrate that they hold a degree without disclosing their grade point average. A contractor should be able to attest to relevant experience without revealing prior client relationships. A professional should be able to confirm that their peer reputation score exceeds a given threshold without exposing either the precise score or the individual assessors who contributed to it.

Storing credential data on-chain in its raw form resolves the fragmentation and fraud problems identified in Section 1.1 but creates a new and equally serious problem: permanent, irrevocable disclosure. Once credential details are recorded on the blockchain, they cannot be expunged. Every verifier whether a legitimate employer or an unauthorised data aggregator retains permanent read access to every stored field. This outcome is incompatible with the principle of data minimisation mandated by modern data protection frameworks (GDPR, 2016, Art. 5(1)(c)) and with the broader expectation of individual autonomy over personal information.

Storing credentials off-chain for instance on a distributed file system such as IPFS (Benet, 2014) and recording only a cryptographic commitment hash on-chain partially addresses this concern by rendering data access conditional. However, hash commitments alone are insufficient for selective disclosure. In order for a verifier to validate a unique attribute assertion, such as confirming that an applicant has a GPA above a certain threshold, either the verifier needs to have access to the entire underlying credential record, or has to rely solely on the assertion of the holder without independent validation.

This is the privacy paradox at the core of blockchain-based identity systems. The properties that make a distributed ledger an effective platform for tamper-evident credential storage transparency and universal read access are opposite to the selective disclosure and privacy that meaningful identity management requires. Resolving this paradox is the main technical challenge addressed by this thesis.

1.5 Zero-Knowledge Proofs as the Resolution Mechanism

Zero-knowledge proofs (ZKPs) provide a mathematically rigorous resolution to the privacy paradox described above. A zero-knowledge proof is a cryptographic protocol by which a prover can convince a verifier that a given statement is true without conveying any information beyond the truth of that statement (Goldwasser, Micali & Rackoff, 1985). The foundational properties of a ZKP completeness (honest provers always succeed), soundness (dishonest provers cannot

fabricate valid proofs), and zero-knowledge (the verifier learns nothing beyond the fact of validity) make it uniquely suited to selective credential disclosure.

Applied to credential verification, ZKPs enable a credential holder to prove attribute claims of the form 'my GPA exceeds 3.5', 'I have accumulated more than three years of relevant work experience', or 'my peer reputation score is above the 70th percentile' without disclosing the underlying GPA value, the specific employment history, or the identities of individual assessors. The verifier receives a compact cryptographic proof that is computationally either valid or invalid; no intermediate data is disclosed in either case.

Modern ZKP constructions specifically the Groth16 variant of zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) (Groth, 2016) produce proofs that are compact (3 group elements, approximately 256 bytes on the BN128 curve), computationally efficient to verify, and compatible with the Ethereum Virtual Machine through native elliptic curve precompiles introduced in EIP-196 and EIP-197 (Reitwiessner, 2017; Buterin & Reitwiessner, 2017). This combination of compactness and EVM compatibility makes on-chain proof verification both practically feasible and economically viable.

The ZKP construction employed in this thesis is as follows. Credential attribute claims are encoded as arithmetic circuits written in Circom 2 (Bellés-Muñoz et al., 2022), a domain-specific language for zero-knowledge circuit design. Each circuit encodes the constraint that a claim holds for example, that the weighted average of a score array exceeds a specified threshold together with commitment checks that cryptographically bind the proof to specific on-chain data. A trusted setup ceremony produces the corresponding proving and verification keys. Credential holders generate Groth16 proofs using the snarkjs library (iden3, 2020) and submit them to an on-chain verifier contract, which employs the EVM's bn128 elliptic curve precompile to verify each proof at a gas cost of approximately 280,000 units a feasible overhead for a high-value attestation in an institutional identity context.

An important practical consideration is that ZKP generation can be rendered entirely invisible to end users. The computational work of constructing a proof which may require several seconds on a standard client device can be handled by the frontend application or a supporting background service without requiring any cryptographic interaction from the user. A credential holder experiences only a brief processing delay since the underlying mathematics are fully abstracted. This design is based as zero-knowledge transparency: the principle that cryptographic privacy mechanisms should be operationally invisible to the users they protect. It is advanced here as a necessary condition for the practical adoption of ZKP-based identity systems by non-technical professional users.

The combination of SBTs and ZKPs enables an identity architecture that achieves what prior approaches have individually been unable to deliver: credentials that are cryptographically immutable and non-transferable, selective disclosure of specific attribute claims without full data exposure, and granular user-controlled access to the underlying credential records all within a single coherent system.

1.6 Research Questions

The above analysis results in the formulation of three formal research questions, which are interrelated: RQ1 establishes the design requirement for privacy-preserving technology; RQ2 investigates if the proposed ZKP mechanism can achieve the performance necessary for achieving the required privacy-preservation design requirement; and RQ3 addresses the broader system design question of structuring and governing access to credential data.

RQ1.

How can a decentralised identity platform based on soulbound tokens protect user privacy while maintaining the cryptographic verifiability of professional credentials?

This question relates to the privacy paradox introduced in Section 1.4. A satisfactory answer will show that not only do ZKPs exist in theory, but also that they can be integrated in practice with SBTs through a deployed system that has operating circuits, an on-chain verifier smart contract and a user experience that does not present the end user with the complexities of cryptography.

RQ2.

Can zero-knowledge proof systems provide efficient and user-transparent credential verification at scale on an EVM-compatible blockchain?

This question addresses the practical feasibility of the ZKP-based verification mechanism. A satisfactory answer requires empirical data: proof generation times, circuit constraint counts, on-chain verification gas costs, and a comparison against alternative approaches where applicable. 'User-transparent' implies that ZKP generation must be operationally invisible to the credential holder; 'efficient' requires that gas cost and latency fall within bounds acceptable for real-world institutional deployment.

RQ3.

What is the appropriate design balance between on-chain transparency and selective disclosure in a professional identity system, and what access control mechanisms can mediate this balance?

This question addresses the governance and system design dimension of the problem. Not all credential data warrants the same level of confidentiality: some attributes are appropriately public, others accessible only to explicitly approved parties, and still others verifiable only in aggregate or threshold form. A satisfactory answer requires a principled access control architecture, the bitmask permission scheme and request-approve-deny lifecycle implemented in this thesis together with an analysis of its correctness properties and practical limitations.

1.7 Scope and Contributions

This thesis presents the design, implementation, and evaluation of SoulBound Identity: a decentralised professional identity platform constructed on the following technology stack a permissioned Hyperledger Besu QBFT blockchain (Chain ID 424242); three Solidity smart contracts (SoulboundIdentity, CredentialsHub, SocialHub); five Circom 2 arithmetic circuits with Groth16 proving keys; and a React 18 frontend application.

The primary contributions of this work are as follows.

A complete proof-of-concept implementation of an SBT-based professional identity platform integrating ZKP credential verification within a single coherent system. To the best of the author's knowledge, no existing deployed platform currently combines these mechanisms in this way.

A five-circuit ZKP subsystem covering degree category, GPA threshold, work experience duration, certification domain membership, and reputation score threshold each with Groth16 proofs verified on-chain by dedicated verifier contracts.

A bitmask access control architecture encoding per-credential-type access grants as a single uint256 bitmask, enabling efficient batch permission operations and a clearly defined request-approve-deny lifecycle.

An empirical evaluation of ZKP performance metrics, including proof generation time, circuit constraint counts, and on-chain verification gas costs in an EVM deployment context, providing a reference baseline for practitioners designing comparable systems.

A user experience design that renders ZKP proof generation operationally invisible to end users, contributing a practical implementation template for zero-knowledge transparency in consumer-facing blockchain applications.

Scope Limitations

Several important limitations bound the scope of this work. The system is a prototype deployed on a private consortium network and is not intended for immediate production deployment. IPFS integration is simulated Content Identifier (CID) generation is implemented, but real distributed file pinning is not. The trusted setup for the ZKP circuits employs a publicly available Powers of Tau ceremony appropriate for a research prototype, but not for a production deployment, which would require a formal multi-party computation ceremony. Proof generation is currently performed client-side in the browser while server-side proof generation, which would substantially improve scalability and mobile performance, is identified as a direction for future work.

1.8 Research Methodology

This thesis utilizes a research method design (Henver et al., 2004), in which it outputs a deployed proof-of-concept system that has been evaluated by explicitly defined design criteria. The research consists of six phases, which work in serial order and provide input to subsequent phases.

Literature Review.

The first phase focuses on the academic and technical foundations required to frame the research problem and justify design decisions. This encompasses the decentralised identity landscape (W3C DIDs, Verifiable Credentials, Self-Sovereign Identity), the conceptual and standardisation history of Soulbound Tokens (Buterin, 2022a; Weyl, Ohlhaver & Buterin, 2022; ERC-5484), zero-knowledge proof constructions (Groth16, PLONK, zk-STARKs), the Circom 2

circuit model, and five deployed or proposed related systems. At the end of Section 2.6 a Structured Identification of the Research Gap that framed the design decisions for the development of the proposed system will be presented.

System Design.

The second phase translates the research gap into a concrete architecture. A threat model is established and a set of formal design goals such as privacy, verifiability, non-transferability, user control, and censorship resistance, are put in place. Providing a stable basis against which design decisions can be evaluated. The three-layer architecture (Blockchain Layer, ZKP Layer, Application Layer), the credential data model, the bitmask access control scheme, and the ZKP circuit selection rationale are all defined at this stage, prior to any implementation. Key trade-offs include: bitmask versus per-grant access, in-browser versus server-side proof generation, on-chain versus off-chain credential storage and are explained in detail in Chapter 3.

Smart Contract Development.

The third phase implements the three core smart contracts (SoulboundIdentity, CredentialsHub, SocialHub) and the on-chain Groth16 verifier contracts on a local Hyperledger Besu QBFT network. The development of each contract follows an iterative cycle of implementation, local deployment (using Hardhat), and functional testing against the access control and credential management requirements. During this phase, gas optimization techniques such as, custom error types, use of EnumerableSet and reentrancy guards, were applied. Contract addresses and deployment parameters are logged to ensure every contract can be reproduced.

ZKP Implementation.

The fourth phase designs, compiles, and tests the five Circom 2 arithmetic circuits (gpa_threshold, work_experience_threshold, degree_category, cert_domain, reputation_threshold). Each circuit is compiled to an R1CS and a WebAssembly witness generator, subjected to a trusted setup using the Hermez Powers of Tau transcript, and tested with snarkjs for witness generation correctness and proof validity. Following the process, the resulting proving and verification keys are deployed and their associated on-chain verification gas costs are measured.

Frontend Integration.

The fifth phase focuses on developing the React 18 frontend application and integrates it with the deployed contracts via ethers.js v5.7.2. The MetaMask network auto-configuration, credential submission flows, access control lifecycle interface, and in-browser ZKP proof generation pipeline (loading .wasm and .zkey artefacts, invoking snarkjs Groth16 prove, submitting the proof to the on-chain verifier) are all implemented and verified against the zero-knowledge transparency design goal, which is that ZKP generation must remain operationally invisible to users.

Evaluation.

The sixth phase evaluates the completed system against the three research questions. Functional evaluation covers five end-to-end user journeys. Quantitative evaluation collects gas costs for all significant operations, browser-side proof generation times, circuit constraint

counts, and .wasm and .zkey file sizes. A security analysis examines access control correctness, replay attack resistance, and rate-limiting behaviour. The results are presented in Chapter 6 while the appropriate responses to the three research questions can be found in Section 6.6.

Chapter 2-Background and Related Work

This chapter surveys the technical and academic foundations that a SoulBound Identity is built with. Section 2.1 examines the decentralised identity landscape, covering the W3C standards for Decentralised Identifiers and Verifiable Credentials and the broader Self-Sovereign Identity movement. Section 2.2 focuses on the conceptual origin of SBTs, the ERC-5484 standard, and existing implementations. Section 2.3 provides a technical grounding in zero-knowledge proof systems, comparing the principal proof constructions and explaining the selection of Groth16 for this project. Section 2.4 describes the blockchain infrastructure on which the system is deployed. Section 2.5 surveys five deployed or proposed systems related to decentralised identity. Finally Section 2.6 combines the elements above into a well-defined statement that outlines the research gap that SoulBound Identity attempts to address and how the thesis contributes towards closing it.

2.1 The Decentralised Identity Landscape

2.1.1 The Centralised Identity Problem

Currently, centralised identity providers dominate the digital identity infrastructure. A professional's credentials are fragmented across institutional silos: a university's student record system, an employer's HR platform, a government e-ID database, a social network's profile store. Each of these are managed by a separate organisation with its own data model, access and retention policy. The absence of interoperability between these silos means that any identity claim that crosses these organizational boundaries whether it is a job application, a professional licence verification, an academic credential check, they all require contacting the original issuing institution directly. Reintroducing exactly the friction and latency that digital systems are intended to eliminate.

This architecture creates three structural deficiencies. First, it is inefficient; high-assurance verification latency is measured in days, not seconds. Second, it is unable to protect an individual's privacy; verifiers typically receive complete credential records when they may need only to confirm a single attribute. Third, it creates single points of

failure and control; if an institution's record system is taken offline, hacked, or shut down, the individual has no means of independent recovery.

2.1.2 Self-Sovereign Identity

The design of Self-Sovereign Identity (SSI) is based on the belief that people should have the ability to manage and control their own identity data. According to Allen (2016), there are ten basic principles of an SSI. A few of them include: existence (users must have an independent existence beyond any platform); control (users must control their identities); access (users must have unrestricted access to their own data); and persistence (identities must be long-lived and not dependent on any single organisation). These principles contrast with the federated Identity model used by most major platforms today (such as OAuth social login), where the platform (or Provider) retains ultimate authority over the Digital Identity.

SSI implementations conventionally separate into three distinct roles: the issuer (an institution that attests to a claim about a subject), the holder (the individual who receives, stores, and presents credentials), and the verifier (a party that needs to confirm a specific claim). Within this model the holder mediates all interactions, presenting credentials selectively to verifiers without routing the request through the issuer (W3C, 2019). This issuer-holder-verifier triangle of trust constitutes the architectural foundation on which both the W3C Verifiable Credentials standard and SoulBound Identity are constructed.

2.1.3 W3C Decentralised Identifiers (DIDs)

A Decentralised Identifier (DID) is a type of persistent identifier that is globally unique, does not require a centralised registration authority and is cryptographically verifiable (Sporny et al., 2022). A DID takes the form of a URI: did:<method>:<method-specific-id>, where the method specifies how the DID resolves to a DID document. The DID document is a JSON-LD document that contains the public keys, authentication methods, and service endpoints of the identifier. For example, did:ethr:0xf1FB... is an Ethereum-based DID resolved by querying the associated address on-chain.

The DID Core specification (W3C, 2022) defines the data model, URL syntax, and resolution requirements without mandating any particular underlying infrastructure. This method-agnostic design allows DIDs to be anchored on blockchain networks, distributed hash tables, or domain names, making them compatible with a broad range of deployment contexts. This design choice is intentional in order to separate the identifier layer from the storage and consensus layer beneath it.

DIDs provide the persistent and user-controlled identifier needed for a Self-Sovereign Identity. However, a DID alone is not a credential; it is used for identifying subjects but it does not attest to any claims about that subject. That is where Verifiable Credentials come into play.

2.1.4 W3C Verifiable Credentials (VCs)

The W3C Verifiable Credentials Data Model (Sporny et al., 2019) defines a standard for expressing cryptographically verifiable claims. A Verifiable Credential (VC) is a JSON-LD document containing a set of claims made by an issuer about a subject that are identified by a DID along with the issuer's digital signature. A Verifiable Presentation (VP) is a derived document in which the holder packages one or more VCs, potentially with selective disclosure, to a verifier.

VCs represent a significant improvement over traditional credential formats: they are machine-readable, cryptographically tamper-evident, and carry the issuer's signature in a standard format that any conforming verifier can validate independently. However, the VC specification does not define transport mechanisms, storage layers, or privacy-preserving disclosure protocols. Therefore, it is a data model not a complete identity system. Most VC implementations store credentials in a digital wallet on the user's device, which creates questions regarding wallet availability, recovery, and cross-device access that the standard deliberately leaves to implementors.

SoulBound Identity adopts the conceptual SSI triangle and the VC issuer-holder-verifier role model, but replaces the wallet-centric off-chain storage model with on-chain credential storage secured by smart contract access control. The detailed reasoning for the design choice, including the associated tradeoffs, will be discussed in detail in Chapter 3.

2.2 Soulbound Tokens

2.2.1 Conceptual Origin

The concept of a 'soulbound' token was introduced by Vitalik Buterin in a January 2022 blog post (Buterin, 2022a), drawing an analogy from massively multiplayer online role-playing games in which certain powerful items, once equipped, are permanently bound to a character and cannot be traded or transferred to other players. According to Buterin, the NFT ecosystem during 2021–2022 revolved around the idea of financially tradable, transferable assets. The key examples of NFTs that fell into this category were Profile Pictures, Digital Art and In-game Items. This focus omitted an entire class of socially significant objects: attestations, achievements, memberships, and credentials whose value is inseparable from the identity of the holder.

A university degree is not valuable because it can be sold; it is valuable precisely because it cannot be legitimately transferred to another person. The same principle applies to professional licences, employment records, and peer endorsements. If these credentials are represented as transferable tokens, they become economically tradable, therefore susceptible to forgery by proxy and so the credential can be purchased rather than earned. Binding a token permanently to a wallet address eliminates this vulnerability: the token's presence in a wallet constitutes evidence that the wallet's controller legitimately received the attestation through the process that generated it.

2.2.2 Decentralised Society

Weyl, Ohlhaver, and Buterin (2022) expanded upon the Soulbound concept by examining it in the context of a larger Sociotechnical System in their publication, " Decentralized Society: Finding Web3's Soul.". The paper proposes a world in which 'Souls' (wallet addresses holding a rich collection of Soulbound Tokens representing diverse social relationships) serve as the foundation for a decentralised social graph.

Through the use of SBTs users can store information regarding their Educational Credentials, Employment History, Community Memberships, and Peer Endorsements, resulting in a Social Graph that can be used for New Types of Applications, including Undercollateralized Lending powered by Social Trust, Quadratic Voting weighted by Community Membership, and Governance in Decentralized Organizations that cannot be captured by those who have Wealth.

The DeSoc concept presents not only an Identity System but also provides a broader Sociotechnical perspective in its design. The authors explicitly acknowledge significant unresolved problems, three of which are directly relevant to this thesis: privacy (a naive SBT implementation exposes the social graph in full on a public chain), recovery (loss of a Soul wallet is irrecoverable without off-chain backup mechanisms), and discrimination risk (immutable on-chain records could be exploited to deny opportunities on the basis of past associations or credentials). This thesis addresses the privacy problem through ZKP-based selective disclosure and the access control problem through a granular per-credential permission system; the wallet recovery and discrimination problems are identified as directions for future work in Chapter 7.

2.2.3 The ERC-5484 Standard

ERC-5484 (Cai, 2022) is the Ethereum Improvement Proposal that formalises Soulbound Tokens as an extension of the ERC-721 NFT standard. Its primary technical contribution is the burnAuth mechanism, which is an enumerated value fixed at minting time that determines which parties hold authority to destroy the token.

burnAuth Value	Authorised Burn Party
IssuerOnly	The address that originally minted the token
OwnerOnly	The token holder (wallet address to which the token is bound)
Both	Either the issuer or the owner
Neither	The token is permanently non-destroyable

ERC-5484 overrides the ERC-721 transfer functions to revert unconditionally, enforcing non-transferability at the protocol level rather than relying on application-layer conventions. A single

Issued event is emitted at minting to provide a canonical record of issuance. The standard intentionally omits credential metadata format, access control policy, and credential semantics. These are delegated to implementing contracts, consistent with ERC-721's design philosophy of minimal core standardisation.

SoulBound Identity implements ERC-5484 in SoulboundIdentity.sol, extending it with a bitmask access control system, rate-limiting, and IPFS metadata storage. The contract enforces a constraint of one token per address at the contract level, a design choice discussed further in Chapter 3.

2.2.4 Existing SBT Implementations

There have been few actual deployments of SBTs that have recently been launched. One of these Passport XYZ (formerly Gitcoin Passport), is an identity verification protocol that issues on-chain attestations representing the completion of identity verification checks (termed 'stamps'), spanning web2 activity, web3 activity, biometrics, and proof-of-personhood verifications (Passport XYZ, 2024).These stamps are aggregated into a composite trust score to classify addresses and resist Sybil attacks across decentralised applications, including quadratic funding rounds. Binance Account Bound (BAB) tokens (Binance, 2022) are SBTs issued to users who have completed Know-Your-Customer (KYC) verification on the Binance exchange, used to gate access to on-chain features requiring verified identity.

However, neither implementation combines SBTs with ZKP-based selective disclosure, and no existing deployed system has been identified that implements granular per-credential access control alongside non-transferable tokens which are the two properties that constitute the primary technical contribution of this thesis.

2.3 Zero-Knowledge Proofs

Having established the identity and token foundations, this section provides the cryptographic grounding for the zero-knowledge proof (ZKP) subsystem. Zero-knowledge proofs are the mechanism by which SoulBound Identity resolves the privacy paradox identified in Chapter 1: enabling credential holders to prove specific attribute claims without disclosing the underlying credential data.

2.3.1 Foundational Propertie

A zero-knowledge proof (ZKP) is an interactive cryptographic protocol between a prover and a verifier. The foundational definition was established by Goldwasser, Micali, and Rackoff (1989), who introduced the concept of knowledge complexity and formally defined the zero-knowledge property. Subsequent work by Goldreich, Micali, and Wigderson (1991) has shown that ZKPs can exist for all NP statements, establishing the theoretical universality of the ZKP paradigm. Any ZKP system must satisfy three distinct properties:

- Completeness. If a statement is true and both parties follow the protocol honestly, the verifier will accept the proof.
- Soundness. If a statement is false, no dishonest prover can convince an honest verifier to accept the proof, except with negligible probability bounded by a security parameter.

- Zero-knowledge. If a statement is true, the verifier learns nothing beyond the validity of the statement itself. Formally, everything computable from the proof transcript can be simulated by a probabilistic polynomial-time algorithm without access to the prover's secret witness. This is known as the simulation indistinguishability condition.

Together these three properties make ZKPs particularly well-suited for credential verification: completeness guarantees that legitimate holders can always produce valid proofs; soundness prevents fraudulent claims from being proven; and zero-knowledge ensures that no sensitive information beyond the specific attested claim is disclosed during the verification process.

2.3.2 zk-SNARKs

A Succinct Non-Interactive Argument of Knowledge (SNARK) is a proof system in which the proof is short (succinct), requires no interactive exchange between prover and verifier (non-interactive), and the verifier's acceptance constitutes a guarantee that the prover possesses a valid secret witness satisfying the proved statement (argument of knowledge). The addition of the zero-knowledge property yields a zk-SNARK.

The methods of achieving non-interactivity are two; via the Fiat-Shamir transformation (Fiat & Shamir, 1986) which collapses a multi-round interactive protocol into a single non-interactive proof by replacing the verifier's random challenges with a hash of the public inputs, or through a Structured Reference String (SRS) produced by a trusted setup ceremony. In the SRS model, the prover uses the SRS to generate a proof and the verifier uses a corresponding verification key derived from the same SRS to check it. Because the proof can be verified by any party at any time, this model is essential for on-chain verification, where there is no synchronous communication channel between prover and verifier.

2.3.3 Groth16

Groth (2016) introduced a pairing-based SNARK construction whose proofs consist of exactly three points in an elliptic curve group (two points in G_1 and one point in G_2). This means the size of the proof is approximately 192–200 bytes. Verification requires only three pairing operations, making Groth16 the most compact known pairing-based SNARK in both proof size and verification computational cost. The compactness of the proofs of Groth16 is due to the algebraic structure of bilinear groups and the particular polynomial commitment scheme used by Groth, detailed treatment is beyond the scope of this thesis but may be found in Groth (2016) and Boneh & Shoup (2023).

On the Ethereum Virtual Machine, verification of a Groth16 proof over the BN128 (alt_bn128) elliptic curve costs approximately 280,000 gas. This cost is bounded and predictable due to precompiled contracts available natively in the EVM: EIP-196 provides elliptic curve point addition and scalar multiplication (Reitwiessner, 2017), EIP-197 provides the optimal Ate pairing check (Buterin & Reitwiessner, 2017), and EIP-1108 reduced the gas costs of these operations to their current levels (Cardozo & Williamson, 2019). Therefore, Groth16 is economically viable to verify high-value attestations on-chain, especially where the number of verifications is small in comparison to the lifetime value of the credential.

The principal limitation of Groth16 is its circuit-specific trusted setup: the SRS is generated for a particular circuit by a multi-party computation (MPC) ceremony in which at least one participant must discard their secret randomness (the 'toxic waste'). If all ceremony participants collude, they could forge arbitrary proofs. In practice, large-scale MPC ceremonies such as the Hermez network's Powers of Tau ceremony, which involved over 200 independent participants significantly limit the collusion assumption. As such, the publicly available Hermez Powers of Tau transcript will be used during this prototype; the implications of this choice for production deployment are discussed in §1.7 and Chapter 7.

2.3.4 PLONK and zk-STARKs - Alternatives Considered and Rejected

The project rejected two alternative proof constructions after comparing their benefits and limitations (See Table 2.1).

There are two major differences between PLONK (Gabizon, Williamson & Ciobotaru 2019) and Groth16. PLONK utilizes a single trusted setup that will support for all circuits up to a given maximum size as opposed to having multiple trusted setups for each circuit like Groth16. This provides a major operational benefit when processing multiple circuits at one time. However, PLONK proofs are larger (approximately 500 bytes) and more expensive to verify on-chain, making them less cost-effective for the per-credential, high-frequency verification use case of this system.

zk-STARKs (Scalable Transparent Arguments of Knowledge) (Ben-Sasson et al., 2018) require no trusted setup, the SRS is replaced with a public hash function, eliminating the toxic waste problem entirely. This is a significant trust-model advantage over Groth16. However, the zk-STARK proof sizes are much larger (tens of Kbytes) and EVM-native STARK verifiers impose prohibitive gas costs with no recursive proof aggregation layer. EVM-native STARK verification is an active research area, several projects (StarkWare, Risc Zero) are making rapid progress but it will not be ready for production during the time this system requires a per-credential verification process.

Property	Groth16	PLONK	zk-STARK
Proof size	~200 bytes	~400 bytes	~50–200 KB
On-chain verification (gas)	~280K	~450–500K	Impractical*
Trusted setup	Circuit-specific	Universal	None
EVM precompile support	Yes (BN128)	Partial	No
Circuit language (this project)	Circom 2	Circom 2 / Plonkish	Cairo / Noir

Groth16 is selected for this project on the basis of minimal proof size, lowest on-chain verification cost, full EVM precompile support, and a mature, well-documented Circom 2 toolchain. The circuit-specific trusted setup is a recognised limitation, mitigated in this prototype by the use of the large Hermez Powers of Tau ceremony.

2.3.5 Circom and the Arithmetic Circuit Model

ZKP systems operate over arithmetic circuits: directed acyclic computational graphs in which values are elements of a prime field F_p and the permitted operations are field addition and multiplication. Any computation expressible as a system of polynomial equality constraints

over F_p known as a Rank-1 Constraint System (R1CS) can be proved with a SNARK. The constraint count of a circuit determines the size of the SRS required, the memory utilized to create a witness and, as a result, the time required to create a valid proof via the SNARK.

Circom 2 (iden3, 2022) is a domain-specific language for writing arithmetic circuits that compiles to an R1CS representation and a WebAssembly witness generator. It provides a template system for modular circuit composition and a standard library of common primitives including range checks, binary comparators, and cryptographic hash functions. The compiler produces a .r1cs file consumed by the trusted setup toolchain and a .wasm file used by snarkjs (iden3, 2020) to compute the witness during proof generation. The five circuits implemented in this thesis range from approximately 1,000 to 15,000 constraints; detailed constraint counts are reported in Chapter 5.

2.3.6 The Poseidon Hash Function

SHA-256 and Keccak-256 are standard cryptographic hash functions, that are computationally efficient in general-purpose software but impose an extremely high constraint cost when encoded as arithmetic circuits: SHA-256 requires approximately 27,000 R1CS constraints per invocation, making its use inside ZKP circuits impractical for any circuit that must remain within tractable constraint bounds.

Poseidon (Grassi et al., 2019) is a hash function designed specifically for the arithmetic circuit model. It works over elements in a prime field and uses the HADES permutation design. This approach alternates between full S-box rounds, where every element is exponentiated in the field F_p, and partial S-box rounds, where only one element is exponentiated. This structure is used to maintain strong cryptographic security while keeping the number of constraints as low as possible. A Poseidon hash over two field elements requires approximately 240 R1CS constraints, roughly two orders of magnitude fewer than SHA-256.

In this system, Poseidon is used to compute the scoresCommit public input of the reputation_threshold circuit: a hash of the full array of peer review scores that binds the proof to a specific on-chain snapshot, preventing replay attacks in which a previously valid proof is submitted against a different or stale score state. The full circuit design is discussed in Chapter 5.

2.4 Blockchain Infrastructure

The Ethereum Virtual Machine (EVM) is a stack-based, deterministic, turing-complete runtime that executes smart contracts on the Ethereum network and its compatible chains (Buterin, 2014; Wood, 2014). Smart contracts are deployed as EVM bytecode at a fixed content-addressed location; transactions invoke a contract by sending a message to that address with encoded calldata. The EVM's gas model assigns a cost to every operation, which measures how much computation it uses. This ensures resources are accounted for properly and helps prevent denial-of-service attacks caused by infinite loops or overly heavy computation.

The EVM properties most relevant to this thesis are:

- Determinism: every validating node executes the same transaction bytecode and reaches the same resulting state; no external data sources can be called from within a contract during execution.
- Immutability: once deployed, contract bytecode cannot be modified unless an explicit proxy upgrade pattern is used. This ensures that access control logic and ZKP verification logic cannot be changed after deployment.
- Events: contracts emit logs that are stored in a Bloom filter indexed structure in each block. These events are the main way off-chain applications observe on-chain state changes without directly reading contract storage.
- Precompiles: a set of cryptographic functions, including elliptic curve operations used in Groth16 verification, are implemented as native EVM instructions at fixed addresses. This allows efficient on-chain cryptography while keeping gas costs bounded.

2.4.2 QBFT Consensus and Hyperledger Besu

The system is a private consortium blockchain that operates on Hyperledger Besu with ChainID 424242, implementing the Quorum Byzantine Fault Tolerant (QBFT) consensus method. The QBFT is a deterministic Byzantine Fault Tolerant (BFT) protocol that was derived from Istanbul BFT. QBFT allows for immediate transaction finality; once a block has been committed by the validator set, it can never be reorganized. It can tolerate up to [(n−1)/3] faulty or malicious validators in a network of n validators. This differs from Ethereum mainnet's probabilistic finality, where a transaction is only considered practically irreversible after multiple confirmation blocks.

There are two major benefits to immediate finality as it applies to an identity system. First, any approved access grants or credential additions become effective as soon as they are included in a block, with no risk that a chain reorganisation could undo the decision. Second is that zero knowledge proofs provided to the on-chain verifier contract will be usable the instant they are submitted, and the validity of the claim can be trusted without further delay while awaiting confirmation again. Together, these properties simplify the system's trust assumptions compared to a deployment on a probabilistically finalised public blockchain.

Hyperledger Besu is a production-grade Ethereum client by the Hyperledger Foundation as a production level solution and as such supports both public Ethereum Mainnet, and Private/Permitted/Enterprise Blockchain networks. The way it facilitates QBFT consensus, provides JSON-RPC API compatibility with established Web3 toolkits like MetaMask, ethers.js, and Hardhat, and includes a configurable permissioning model. These features make it well suited for a prototyped application that requires standard functionality but runs on a private network.

2.4.3 OpenZeppelin Contract Patterns

OpenZeppelin Contracts (OpenZeppelin 2023) is the leading open-source collection of Solidity development tools for developing secure smart contracts using audited designs and shared security tools. The three OpenZeppelin components used within this application include:

- Ownable2Step: a two-step ownership transfer pattern that requires the new owner to explicitly accept the transfer before it becomes effective. This prevents accidental or malicious ownership transfers from leaving the contract without an accessible administrator.
- ReentrancyGuard: a mutex that prevents reentrant calls to guarded functions, protecting state-changing operations against the class of reentrancy attacks demonstrated by the 2016 DAO exploit.
- EnumerableSet: a library that provides address and bytes32 sets with O(1) add, remove, and membership-check operations, along with full enumeration support. It is used for token to owner mappings and approved requester tracking.

The project also applies OpenZeppelin's custom error pattern (introduced in Solidity 0.8.4) as opposed to require statements with string messages. This constitutes reduced gas costs for revert operations and improved off-chain error introspection, as custom errors carry a 4-byte selector that can be decoded by ABIs rather than requiring string parsing.

2.5 Related Systems

This section reviews five existing systems that are most relevant to SoulBound Identity. The systems range from zero-knowledge proof-based frameworks to more traditional centralized verification platforms. The systems will be analysed based on six criteria, as outlined in the gap analysis provided in Section 2.6.

2.5.1 Polygon ID

Polygon ID (Polygon, 2022) is an identity framework developed by Polygon and built on the iden3 protocol. Credentials are issued off-chain as W3C Verifiable Credentials and stored in the user's mobile wallet. A commitment to each credential is stored in the holder's identity Merkle tree, where the root is stored on-chain in the Polygon PoS blockchain. There are ZKP circuits, implemented in Circom 2 with Groth16 proofs, and allow users of Polygon ID to prove claims regarding their credentials, such as age (e.g., being over 18) or belonging to a group, without directly revealing the credential contents.

Polygon ID is the closest existing example of an identity system that utilizes a technology stack and has similar overall design objectives to SoulBound Identity. However, the two systems differ in three significant areas. First, Polygon ID stores credentials off-chain in a mobile wallet, whereas SoulBound Identity stores credentials on-chain with smart contract based access control. This design prioritizes availability and permissionless verifiability rather than relying on wallet storage. Second, Polygon ID does not include a soulbound, non-transferable identity anchor, since its credentials exist as VC documents that could theoretically be copied or shared. Third, Polygon ID does not provide a peer reputation mechanism or an on-chain social graph. While Polygon ID is designed for general-purpose identity verification with enterprise issuer integrations, SoulBound Identity focuses specifically on professional credentials combined with integrated social attestation.

2.5.2 Worldcoin / World ID

The biometric identification system developed by Worldcoin Foundation (Worldcoin Foundation, 2023), known as Worldcoin (now rebranded as World), uses a specialized device called "The Orb," which performs iris scans to create a unique iris identifier (IrisCode) for each user (meaning that two different users will never have the same IrisCode). At the time of writing, the IrisCode was added to the Merkle tree and subsequently users could utilize a zero-knowledge proof protocol based on Semaphore (Gurkan et al., 2021) to prove they are a member of the collective (all registered humans), while at the same time not revealing either their IrisCode or tying together proof of action back to one particular individual; that resulting on-chain artifact is called a World ID.

The mission of the Worldcoin Foundation is narrowed specifically to an issue that is Sybil-resistant for public digital goods. They do not deal with the credentialing of individuals for their profession, determine access control or tie reputation systems to any one individual. However, the reliance on a proprietary iris code database and proprietary hardware introduces considerable issues regarding ethics, privacy, or regulatory compliance. Such issues do not exist with the SoulBound Identity trust model because it does not use biometry or require specialized hardware.

2.5.3 Civic

Civic Technologies (Civic Technologies, 2018) is an identity verification platform that issues reusable KYC attestations. Users complete identity verification, including document scanning and liveness detection, through Civic's partner network. The verified status is then stored in Civic's Secure Identity Platform and can be presented to third parties via QR code or API calls. When utilizing this type of architecture, the blockchain is used primarily as an append-only audit trail for verification events rather than the main layer for credential storage or access control.

Civic Technologies illustrates a centralized verification and blockchain anchoring hybrid model. The verifiable credentials provided through Civic technologies are not verifiable by any other means than those outlined above, and they are not open to all parties to verify. This contrasts directly with SoulBound Identity's design goal of permissionless verifiability, where any party with access to the blockchain and the on-chain zero-knowledge proof verifier contract can validate a proof without needing to route through the original issuing institution.

2.5.4 MIT Digital Certificates (Blockcerts)

The MIT Media Lab's Digital Certificates project (Nazaré, J.,Duffy, K. H.,& Schmidt, J. P. (2016)), subsequently generalised as the Blockcerts open standard (Schmidt, J. P. (2016). *Blockcerts*), issues academic credentials as signed JSON-LD documents anchored on the Bitcoin or Ethereum blockchain. The credential itself is delivered off-chain to the recipient as a file or URL, whereas the blockchain record contains a Merkle root of a batch of issued credentials, allowing any individual credential's validity to be verified against the on-chain root without exposing the credential contents of the batch.

The Blockcerts standard was a key step in establishing an architecture for blockchain-based credentialing which allows for complete verification of the credential without relying on a centralized verification service. That being said, Blockcerts credentials cannot be selectively

disclosed to verifiers, meaning a verifier who receives a credential file gains access to all included fields. There is no zero-knowledge proof layer, no access control mechanism, and no concept of non-transferability; a Blockcerts credential is as easily copyable and shareable as a PDF document.

2.5.5 LinkedIn Credential Verification

LinkedIn has introduced credential verification features through partnerships with third-party verifiers, including CLEAR and Persona, allowing users to display a verified badge on their education and employment profile entries (LinkedIn, 2023). In order to get verified, users must submit information to a third-party verification service that includes government-issued identity documents or institutional login credentials. The third-party service will confirm the user's claim and inform LinkedIn to display the verification badge on the user's profile.

LinkedIn's verification model is a snapshot of the currently accepted model for professional identities: Ad hoc, platform dependent, centralized verification process with no cryptographic assurance, limited to the platform, no user management of downstream data exchange, no privacy preserving, and no credential portability outside of LinkedIn. Thus, LinkedIn's verified badge carries weight based on the users' confidence in LinkedIn and its verification partners. It has no independent means of verification or validation outside of the LinkedIn platform and is not verifiable by external systems.

2.6 Research Gap

This chapter provides an overview of the survey results indicating a distinct and consistent shortfall of the existing decentralized identification environment. The five systems reviewed plus SoulBound Identity were positioned on a table (Table 2.2) relative to six design criteria that provide together towards defining the characteristics of the target system.

System	Non-transferable tokens	ZKP selective disclosure	Granular access control	On-chain credential storage	Peer reputation	Open / permissionless
Polygon ID	No	Yes	No	No (root only)	No	Partial
Worldcoin	No	Yes (Semaphore)	No	No	No	Partial
Civic	No	No	No	No	No	No
Blockcerts	No	No	No	No (root only)	No	Yes
LinkedIn Verif.	No	No	No	No	No	No
SoulBound Identity	**Yes (ERC-5484)**	**Yes (Groth16)**	**Yes (bitmask)**	**Yes**	**Yes**	**Yes**

The table reveals that no existing system satisfies simultaneously all six criteria. The gap comprises four specific dimensions, in order of their prominence to the contribution of the thesis:

1. Simultaneous use of SoulBound Tokens (SBT) and Zero-Knowledge Proofs (ZKP). Polygon ID employs ZKPs but not soulbound tokens and although existing SBT

deployments such as Gitcoin Passport and BAB use non-transferable tokens they provide no ZKP based selective disclosure. No deployed system combines both mechanisms. This represents the primary technical gap that SoulBound Identity addresses.

2. Granular access control over on-chain credential data. When credentials are stored on-chain, they are either fully public or not stored on-chain at all. No existing system implements a fine grained, user-controlled access request, approve, and deny lifecycle over on-chain credential attributes. The bitmask permission architecture described in Chapter 3 directly addresses this gap.
3. Integrated peer reputation with ZKP attestation. Peer reputation systems exist on centralized platforms such as LinkedIn endorsements and StackOverflow scores, but none integrate with on-chain identity or expose ZKP verified threshold proofs of reputation scores. SoulBound Identity's SocialHub contract and reputation threshold circuit fill this gap.
4. A unified, coherent platform. Existing systems address at least one of the above systems independently and thus require the credential holder to manage credentials across multiple protocols and systems. By integrating SBT anchored identity, ZKP credential verification, and peer social reputation into one platform with a common user interface, SoulBound Identity creates a single solution to eliminate coordination burdens on credential holders.

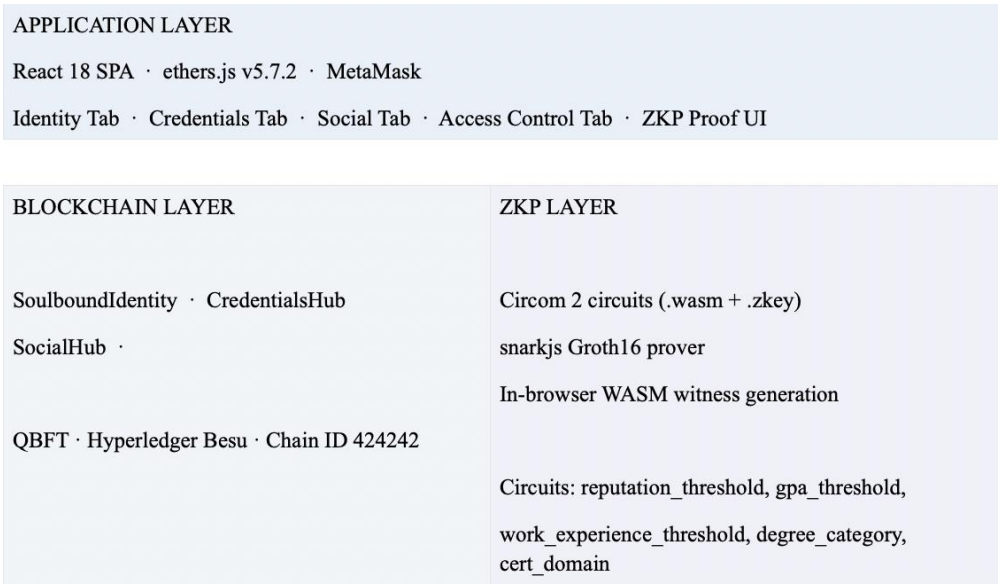
This gap is the basis for the research conducted in this thesis. The chapters that follow describe the system design (Chapter 3), implementation (Chapter 4), ZKP subsystem (Chapter 5), and empirical evaluation (Chapter 6) of a platform addressing all four areas presented above.

Chapter 3 - System Architecture and Design

This chapter presents the architecture and design of SoulBound Identity. The primary focus is entirely with why the system is structured as it is: the threat model that shaped the design, the responsibilities and boundaries of each component, the data models and access control schemes, and the key trade-offs that were explicitly decided during the design process. Interface definitions will be provided where they are needed to describe the boundaries of the various components, while implementation detail will be discussed in Chapter 4.

3.1 Architecture Overview

SoulBound Identity is built on a 3-layer system comprised by: a Blockchain Layer that provides cryptographically secured storage, smart-contract logic, and on-chain ZKP verification, (the ZKP Layer handles circuit proving and the generation of cryptographic attestations). Finally, an Application Layer that provides the user interface and mediates all interactions between the user and the two layers below it.



It is also important to note that the separation of the ZKP Layer from the Blockchain Layer is an intentional design choice. ZKP proof generation is a heavy computational and stateless process that creates the fixed-size output of a ZKP known as the Groth16 proof. By treating it as a distinct layer that is called from the application layer and submitting its output to the blockchain layer, the design allows the proof generation substrate to evolve independently (from browser-side WASM to server-side Node.js, for example) without requiring any changes in the smart contract or front-end components.

The three layers communicate via two interfaces. The application layer communicates with the blockchain layer through the JSON-RPC API exposed by the Besu node (via ethers.js and MetaMask). The application layer communicates with the ZKP layer by loading .wasm and .zkey files and invoking snarkjs (iden3, 2020) groth16.fullProve() directly in the browser. The proof output is then passed to the blockchain layer through a standard contract call.

3.2 Design Goals and Threat Model

3.2.1 Design Goals

The design of SoulBound Identity is directed by five principal goals, listed in priority order. Where design decisions involve trade-offs between goals, the higher-ranked goal takes precedence.

G1 Privacy by default; All credential data stored in the system must be considered private by default and any credential or social data can only be accessed by the token holder unless the token holder initiates an access grant. Verifiers' ability to check specific claims must allow them to do so without receiving any actual data. The operationalisation of this objective is the principle of data protection by design and by default (GDPR, 2016, Article 25, Cavoukian, 2009).

G2 Cryptographic verifiability; All credential-related claims that can be verified externally must have a way to do so via cryptography, such as an on-chain record from a trusted issuer, or an on-chain-verifiable ZKP. Self-asserted credentials are labelled as unverified, and issuers must be authorized at the smart contract level before their attestations can be considered verified.

G3 Non-transferability; Identity tokens must be permanently bound to one specific wallet address. If Identity Tokens were allowed to be transferred or delegated, it would enable identity fraud (for instance, selling a high-reputation identity, or renting out credentials), which would violate the established trust model for the identity verification system. This goal is achieved with ERC-5484 (Cai, 2022) and is consistent with the primary argument for soulbound tokens put forth by Weyl, Ohlhaver and Buterin (2022).

G4 User control; All access grants, access revocations, and credential additions (for self-asserted credentials) must be initiated exclusively by the token owner. No other party (e.g., contract owner, issuer, or platform operator) can read/modify/grant access to a user's credential data without the owner's permission via the on-chain record. This supports the core principle of user control within SSI (Allen, 2016).

G5 Usability transparency; Cryptographic complexity must be invisible to end users. ZKP generation, IPFS CID management, bitmask encoding, and contract interaction must all be abstracted by the frontend. A user should be able to issue credentials, approve access, and generate ZKP attestations with no knowledge of the underlying cryptography. This characteristic is referred to as zero-knowledge transparency as per Chapter 1, and is required for mass market adoption.

3.2.2 Threat Model

The threat model identifies the main categories of attackers and their capabilities.

Credential forger. An attacker who attempts to claim credentials they do not legitimately hold. The system's defence is the issuer authorisation system in CredentialsHub: only wallet addresses explicitly authorised by the contract owner can issue verified credentials. Self-asserted credentials are immutably flagged as verified = false. A credential forger who self-asserts a false credential produces an on-chain record that is provably unverified. A verifier querying the credential will see the verified flag and draw the appropriate conclusion.

Snooping verifier. An attacker who has been granted access to one credential type and attempts to read credential types for which they have no grant, or who attempts to read another user's data entirely. The bitmask access control system in CredentialsHub and SocialHub enforces per-type access at the contract level; view functions revert with NoAccess() if the caller's bitmask does not include the queried type. No partial read of unauthorised data is possible.

Replay attacker. An attacker who captures a valid ZKP proof and attempts to resubmit it to claim a reputation attestation for a different token, or to claim a threshold attestation that is no longer valid (because the subject's scores have decreased). The scoresCommit public input of the reputation_threshold circuit IS a Poseidon hash (Grassi et al., 2019) of the exact score array which binds the proof to a specific snapshot of on-chain data. A proof generated from an old score array will fail verification if the current on-chain state differs, because the verifier checks the public commitment against the current state.

Spam requester. An attacker who submits a high volume of access requests to harass a token owner or exhaust on-chain storage. The rate-limiting system in SoulboundIdentity enforces a

maximum of 10 access requests per address per day and a 1-hour per-token cooldown. Requests that exceed these limits revert at the contract level, costing the attacker gas without creating any on-chain state.

Malicious issuer. An authorised issuer who attempts to issue false credentials to arbitrary tokens. The issueCredential function requires the caller to be in the authorizedIssuers mapping for the relevant credential type. Authorisation is managed by the contract owner via setIssuerAuthorization. An issuer can be de-authorised at any time; upon de-authorisation, the issuer loses the ability to issue further credentials. Credentials already issued by a de-authorised issuer remain as immutable on-chain records.

Outside the threat model. This system does not claim to protect against a compromised token owner (lost private key or deliberate misbehaviour). Wallet security is the user's responsibility and is out of scope. The system also does not protect against a malicious contract owner (the deployer), who holds administrative privileges including pausing and issuer management. This is a known limitation for a prototype that would require a governance mechanism such as a timelock controller or DAO-governed upgrade proxy in production.

3.3 Smart Contract Architecture

The system is composed of three smart contracts. Each contract has a single, well-scoped responsibility; contracts interact through narrow, explicitly defined interfaces. This structure limits the impact of any individual contract vulnerability to the component in which it is located.

SoulboundIdentity is the identity root. It implements ERC-5484 (Cai, 2022) to issue non-transferable tokens (one per wallet address), stores IPFS CIDs for off-chain metadata, and manages the access request lifecycle (request → approve/deny → expiry/revocation). It is the source of ground truth for token ownership: all other contracts resolve ownership by calling SoulboundIdentity.ownerOf(tokenId). It also implements the rate-limiting logic for access requests.

CredentialsHub is the professional credential registry. It stores five types of credentials (Degree, Certification, Work Experience, Identity Proof, and Skill), all of which are stored in a unified data structure that distinguishes issuer-verified credentials from self-asserted ones. It maintains the bitmask access control scheme for credential type access and the authorised issuer registry. GPA values and domain/category enums are stored on-chain; rich metadata is stored off-chain on IPFS with on-chain bytes32 hash commitments.

SocialHub is the social attestation layer. It stores three categories of social data: peer reputation reviews (0–100 numeric scores with cached running totals), project portfolio entries (IPFS-linked metadata with collaborator lists and status tracking), and skill endorsements (both custom-hash skills and 16 predefined standardised skills with per-endorser duplicate prevention). A separate bitmask scheme controls access to each of the three sections independently.

3.3.2 Contract Interaction Diagram

Dependency relationships are depicted in figure 3.2. It is a deliberately directed and acyclic dependency graph: SoulboundIdentity does not have any dependencies on other system contracts; both CredentialsHub and SocialHub maintain an immutable relationship with SoulboundIdentity for ownership resolution but do not have a dependency on each another. Therefore, this flat structure avoids cross contract re-entrant cycles and eases the assessment of upgradability.

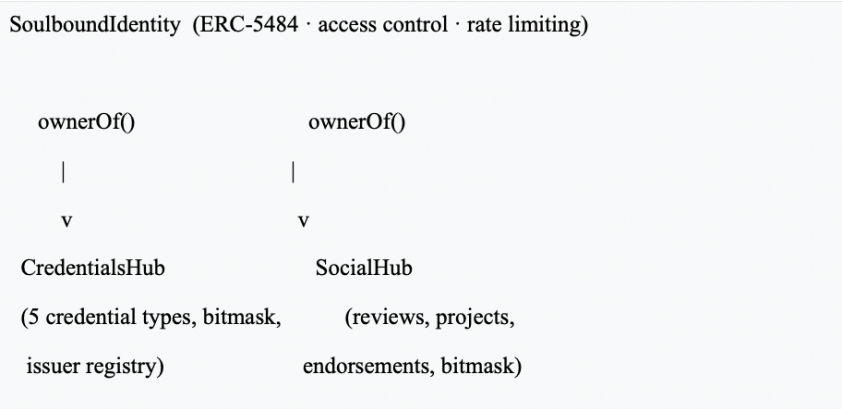


Figure 3.2 — Contract dependency topology. Arrows indicate ownerOf() call direction.

3.3.3 The ISoulboundIdentity Interface

The two satellite contracts interact with the identity root through a minimal interface:

```
interface ISoulboundIdentity {

    function ownerOf(uint256 tokenId) external view returns
(address);

}
```

This deliberate minimalism means CredentialsHub and SocialHub can operate with any ERC-721-compatible identity contract and not just this specific SoulboundIdentity implementation. It also means the interface cannot be exploited by a malicious call since ownerOf is a pure read operation with no side effects.

3.4 Credential Data Model

3.4.1 The Unified Credential Structure

All five credential types share a single on-chain data structure. Table 3.1 enumerates the fields, their Solidity types, and their semantic roles.

Field	Solidity Type	Semantic Role
credentialId	uint256	Globally unique identifier, auto-incremented.
credType	CredentialType	Enum: Degree / Certification / WorkExperience / IdentityProof / Skill.
metadataHash	bytes32	Keccak256 hash of the IPFS CID for off-chain rich metadata.
issuer	address	Issuing address. Token owner for self-asserted; authorised issuer otherwise.
issueDate	uint64	Unix timestamp of issuance. Repurposed as job start date for WorkExperience.
expiryDate	uint64	Unix timestamp of expiry (0 = non-expiring). Repurposed as job end date for WorkExperience (0 = currently employed).
status	CredentialStatus	Enum: Active / Revoked / Expired. Evaluated at query time against block.timestamp.
category	uint8	Multipurpose: SkillCategory for skills; DegreeCategory / CertificationDomain for respective types.
verified	bool	True if issued by an authorised issuer; false if self-asserted. Primary signal for credential consumers.

The verified flag is the single most important field for a credential verifier. A credential with verified = false is a self-assertion: the holder claims to have the credential but no authorised third party has attested to it. A credential with verified = true means an address in the authorizedIssuers mapping for that credential type has signed an Ethereum transaction attesting to the credential. Therefore, a cryptographic guarantee that the issuer participated in the on-chain record creation.

3.4.2 Type-Specific On-Chain Fields

Certain credential types carry additional structured on-chain data beyond the universal struct:

- Degree credentials store GPA as a uint16 scaled by 100 (e.g., 350 = GPA 3.50) and a DegreeCategory enum. On-chain GPA storage is required as the private input to the gpa_threshold ZKP circuit.
- Certification credentials store a CertificationDomain enum on-chain, enabling the cert_domain circuit to prove categorical membership without disclosing the full credential.
- Work Experience credentials use issueDate as the job start date and expiryDate as the job end date (0 = currently employed). The getTotalWorkExperience() view function accumulates duration across all active work credentials, providing the input for the work_experience_threshold circuit.

3.4.3 On-Chain vs. IPFS Off-Chain Split

The design distinguishes between data that must be on-chain for verifiability and data that only needs to be accessible to authorised parties.

On-chain: structured fields needed for ZKP circuit inputs (GPA, category enums, start/end dates), credential status (Active/Revoked/Expired), issuer address, timestamps, and the verified flag.

Off-chain (IPFS): institution names, credential titles, descriptions, document hashes, URLs, and any other human-readable content. Stored as a JSON document at an IPFS CID. The metadataHash field on-chain is the keccak256 hash of the CID string, providing a cryptographically binding commitment to the off-chain content.

This split is motivated by two constraints. First, Ethereum storage is expensive, a new 32-byte slot via (SSTORE) needs 20,000 gas per new slot (Tang, 2019), making on-chain rich metadata economically impractical. Second, privacy: the CID is only exposed to parties with an approved access grant, and the metadataHash is a one-way commitment that reveals nothing about the CID to unauthorised observers.

In this prototype, IPFS pinning is handled via Pinata, ensuring that metadata JSONs are persistently available on the public IPFS network for the lifetime of the pin.

3.5 Access Control Design

3.5.1 The Bitmask Permission Scheme

Access control in SoulBound Identity is carried out using two separate, but complementary mechanisms based on bitmasks: Soulbound Identity uses Bitmasks for access control over the metadata stored on IPFS, and also CredentialsHub/SocialHub use Bitmasks for access control over categories of credential/social data.

In the CredentialsHub, the mapping grantedTypes holds a uint8 bitmask for each (tokenId, viewer) combination, with each bit in the bitmask corresponding to a credential type.

Bit	Decimal Value	Credential Type
0	1	Degree
1	2	Certification
2	4	WorkExperience
3	8	IdentityProof
4	16	Skill

To check whether a caller can access a specific credential type, the contract evaluates:

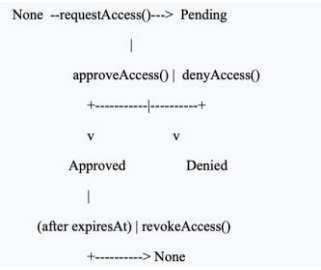
(grantedTypes[tokenId][caller] >> uint8(credType)) & 1 == 1

This single bit-shift and AND operation runs in O(1) and costs approximately 200 gas. A token owner who wants to grant an employer access to degrees and work experience only (bits 0 and 2) sets the bitmask to 0b00101 = 5. The employer can query degrees and work experience but receives NoAccess() for all other types.

The same pattern applies within SocialHub, where bits 0-2 give access rights to Reviews, Projects, and Endorsements respectively. The bitmasking strategies between CredentialsHub and SocialHub are separate; meaning that granting a user access on one application makes no implied grant for the other application.

3.5.2 The Request/Approve/Deny Lifecycle

The access request lifecycle in SoulboundIdentity governs access to the IPFS metadata CID (and through it, the full identity profile). The lifecycle has five states:



A requester who has been denied or whose access has expired returns to None and may submit a new request subject to the cooldown. A requester who is re-approved replaces their expired access record with a new Approved state and expiry timestamp.

The default access grant duration is 30 days (2,592,000 seconds), configurable per approval. After expiry, canView() returns false automatically, there are no on-chain transactions needed to enforce expiry because the check is done at query time against block.timestamp.

3.5.3 Rate Limiting and Cooldown Design

Two forms of rate limiting provide protection against spam access requests.

Per-token cooldown. After a requester makes an access request to a specific token, they are prohibited from making another access request for the same token for one hour (REQUEST_COOLDOWN=1 hours). This helps to prevent an excess of repeated requests directed at a specific token owner on the same day.

Daily global limit. Each address is limited to 10 access requests per calendar day (MAX_REQUESTS_PER_DAY = 10) across all tokens. The daily counter resets at midnight UTC (block.timestamp / 1 days). This prevents an attacker from distributing harassment across many target tokens to circumvent the per-token cooldown.

These limits were chosen based on realistic usage patterns: a legitimate user making access requests to multiple employers in a single day would rarely need more than 10, and re-requesting access to the same token within an hour has no identifiable legitimate use case in a professional credential context.

3.5.4 Batch Operations

Both batchApproveAccess and batchDenyAccess are provided on SoulboundIdentity, and batchGrantCredentialAccess and batchGrantSocialAccess are provided on CredentialsHub and SocialHub respectively. These batch operations allow a token owner to process multiple pending requests in a single transaction, significantly reducing gas costs and the number of MetaMask confirmation dialogs required in practice. A token owner who needs to approve access for ten employers simultaneously would otherwise face ten separate transactions.

3.6 ZKP Subsystem Design

3.6.1 The Role of ZKPs in This System

The ZKP subsystem exists to bridge the gap between two incompatible requirements: (a) credentials must be stored on-chain or have on-chain commitments to be cryptographically verifiable, and (b) credential details must be private. The bridge is a selective disclosure mechanism: a user can generate a ZKP that proves a specific claim holds without revealing the underlying data.

Five circuits address the five highest-value credential claims:

Circuit	Claim Proved	Key Private Inputs	Key Public Inputs
`reputation_threshold`	Average peer score >= T	Score array (50 elements)	Threshold, tokenId, scoresCommit
`gpa_threshold`	GPA >= T	GPA value (scaled x100)	Threshold, tokenId
`work_experience_threshold`	Total experience >= T months	Individual job durations	Threshold, tokenId
`degree_category`	Degree is in category C	Full degree data	Category, tokenId
`cert_domain`	Certification is in domain D	Full cert data	Domain, tokenId

The choice of these five circuits was driven by the most common employer verification queries: qualification in a specific field (degree category and GPA), relevant sector certification (cert domain), sufficient practical experience (work experience), and peer-assessed professional competence (reputation). Each circuit proves a categorical or threshold-based claim, representing the type of query that is useful to a verifier while still preserving the holder's privacy.

3.6.2 Groth16 Selection Rationale

As outlined above, there are five specific properties that were influential in the selection of Groth16 (Groth, 2016) as the proof system to be used. Additional quantitative comparisons with both PLONK and zk-STARKs can be found in Chapter 2, Table 2.1.

1. Proof size. A Groth16 proof contains three elliptic curve points, specifically, two points from G1 and one point from G2. The total size of the proof is around 192–200 bytes in total, making Groth16 the SNARK system with the smallest proof size, thus reducing the cost of storage on the blockchain, as the costs of calldata will be lower when uploading proofs to a blockchain.
2. EVM precompile support. The EVM includes native precompiles for BN128 elliptic curve addition (Reitwiessner, 2017), scalar multiplication (Reitwiessner, 2017), and optimal Ate pairing (Buterin & Reitwiessner, 2017). Groth16 over BN128 exploits all three, reducing on-chain verification cost to approximately 280,000 gas.
3. On-chain verification cost. Among deployed SNARK systems, Groth16 has the lowest on-chain verification gas cost on EVM (see Chapter 2, Table 2.1). For a high-value identity attestation checked once per hiring decision, 280,000 gas is economically acceptable.
4. Circom 2 toolchain compatibility. Circom 2 (iden3, 2022) and snarkjs (iden3, 2020) have mature, production-tested Groth16 support including browser-side WASM witness generation. The toolchain has been validated in production systems including Polygon ID (Polygon, 2022) and the Hermez Network (Hermez Network, 2020).

5. Circuit-specific trusted setup. While the requirement for a circuit-specific trusted setup is a limitation, it is acceptable for a known, fixed set of five circuits. The publicly available Hermez Powers of Tau transcript (over 200 independent participants) is used for this prototype.

3.6.3 The Poseidon Commitment Scheme

For the reputation_threshold circuit, a cryptographic commitment to the score array is required. It serves two purposes: binding the proof to a specific snapshot of on-chain data (preventing replay against outdated scores), and providing a public value the verifier can check against the current state without receiving the scores themselves.

Poseidon (Grassi et al., 2019) was selected as the hash function. As discussed in §2.3.6, Poseidon requires approximately 240 R1CS constraints per two-input hash versus approximately 27,000 for SHA-256. For a 50-element score array, the Poseidon hash chain requires approximately 49 two-input invocations, totalling roughly 12,000 constraints which are feasible within the circuit size constraints of the Powers of Tau ceremony.

The scoresCommit is computed as Poseidon(scores[0], ..., scores[49], reviewCount) and published as a public input. The verifier contract independently computes the same hash from the on-chain score data and checks it against the submitted value. If a mismatch happens, it causes the proof to be rejected.

3.7 Key Design Trade-offs

3.7.1 Bitmask vs. Per-Grant Access Control

An alternative to the bitmask scheme would be a per-grant mapping: a separate mapping(uint256 => mapping(address => mapping(CredentialType => bool))) storing one boolean per (token, viewer, type) triple. This approach is conceptually simpler and avoids encoding and decoding bitmasks.

The bitmask approach was chosen for three reasons. First, gas efficiency: updating all five credential type permissions for a viewer requires a single SSTORE on one storage slot versus five separate SSTORE operations. Second, batch readability: the caller's entire permission profile is read in a single SLOAD and all five type checks performed with bitwise operations without iterating through any logic. Third, compactness: the full permission state fits in a single storage slot as a uint8 value packed into a 32-byte word, whereas the per-grant approach requires five slots per (token, viewer) pair.

The trade-off of using the bitmask approach is that the raw on-chain storage is less readable than the per-grant mapping and requires a minor decode step at the client level. This is an acceptable cost given the gas savings.

3.7.2 In-Browser vs. Server-Side Proof Generation

The prototype implements in-browser proof generation: the .wasm witness generator and .zkey proving key are loaded into the browser, and snarkjs runs the Groth16 prover in a Web Worker. This approach has two significant advantages: it requires no server infrastructure beyond the

blockchain node, and it preserves privacy since the private inputs (score array, GPA value) never leave the user's browser.

The limitations are equally significant. The .zkey files for complex circuits can be tens of megabytes, requiring download on first use. Groth16 proving is computationally intensive; the reputation_threshold circuit with 50 score elements can take 15–60 seconds on a mid-range laptop and is impractical on mobile. The browser JavaScript engine does not have access to native multi-threading optimisations available to a Node.js server.

An alternative design places proof generation on a server-side Node.js process. Since credential data for circuits with entirely public inputs (degree_category, cert_domain, work_experience_threshold) is already on-chain, the server can read inputs directly without introducing any additional privacy cost. For circuits with private inputs (gpa_threshold, reputation_threshold), the holder would need to transmit private values to the server, introducing a trust assumption the in-browser approach avoids. The current in-browser implementation is appropriate for this prototype; a production deployment would benefit from server-side generation for circuits with public inputs.

3.7.3 On-Chain vs. Off-Chain Data Storage

When deciding what to store on-chain or using IPFS, one must consider three trade-offs: cost, privacy and availability.

On-chain storage is expensive: an EVM SSTORE for a new 32-byte slot costs 20,000 gas (Tang, 2019). Storing full credential metadata on-chain would cost hundreds of thousands of gas per credential which is prohibitive for a user with a dozen credentials. IPFS storage is nearly free and can expand to unlimited sizes.

However, IPFS storage requires the CID to be known for retrieval. In the current design, the CID is stored in SoulboundIdentity and revealed only to parties with an approved access grant. A party without access cannot find the IPFS content, because the metadataHash is a one-way commitment.

The availability risk is that if IPFS content is not pinned it may become unavailable. This is mitigated by persistent pinning via a service such as Pinata or a self-hosted node.

The result is the split described in §3.4.3: all data needed for cryptographic verifiability is stored on-chain; all human-readable descriptive data is stored off-chain. This minimises on-chain storage costs while preserving the on-chain data necessary for ZKP circuit inputs.

Chapter 4 – Implementation

This chapter describes the implementation of the architecture presented in Chapter 3 as realised in code. Section 4.1 surveys the technology stack and explains each tool choice. Section 4.2 covers the smart contract implementation, highlighting key engineering decisions with code excerpts. Section 4.3 describes the frontend architecture, its component structure, state management, and MetaMask integration. Section 4.4 documents the deployment

sequence and the deployed contract addresses. The ZKP Circuit Implementation is described in detail in Chapter 5.

4.1 Technology Stack

4.1.1 Blockchain Runtime: Hyperledger Besu with QBFT

The system is deployed on a permissioned Hyperledger Besu network (Hyperledger, 2022), which is operating under the QBFT consensus protocol with Chain ID 424242. Access to this network is provided via a standard JSON-RPC endpoint located at https://rpc.dimikog.org/rpc/. The QBFT protocol was selected over Clique (Proof-of-Authority) and standard Proof-of-Work or Proof-of-Stake due to its ability to provide immediate deterministic finality: Once a block is committed to the chain using QBFT, it cannot ever be changed so that any access grants and credential additions that occur at that time are irreversible.

The Ethereum Virtual Machine (EVM) compatibility of Besu has been tested and proven complete for all purposes of this project: Every feature of Solidity through version 0.8.20 is supported, all OpenZeppelin libraries compile without modifications, and all of the precompiles using the BN128 curve for on-chain verification of Groth16 ((EIP-196, Reitwiessner, 2017; EIP-197, Buterin & Reitwiessner, 2017) are present and functional.

4.1.2 Smart Contract Language and Toolchain

Solidity 0.8.20 is used for all three contracts (Solidity, 2023). Version 0.8.x was chosen for its built-in arithmetic overflow protection, which eliminates the need for the SafeMath library, and for native support for custom errors (added in version 0.8.4) that can significantly reduce the revert gas cost compared to string-based require messages.

Hardhat serves as the development and deployment framework (Nomic Foundation, 2023). Its local Hardhat Network, a fast in-process EVM, was used throughout development for unit testing before deploying to the live Besu node. Hardhat's task system allows for the ability to script the deployment process and automatically capture the addresses that are generated.

OpenZeppelin Contracts v5 provides the audited base contracts for all contracts that are a part of the overall system (OpenZeppelin, 2023): ERC721Enumerable (this will be an extension of the ERC-5484 implementation), Ownable2Step, ReentrancyGuard, Pausable, and EnumerableSet.

4.1.3 ZKP Toolchain

Circom 2, the latest version of an open-source toolset for building zk-SNARK circuits written in Circom language is used to write all five arithmetic circuits (iden3, 2022). Circom 2 introduced a new template system with explicit signal types (input, output, intermediate) to allow for more efficient constraint optimisation than Circom 1. The compiler produces a .r1cs constraint system file, a .wasm witness generator, and a .sym symbol table for debugging.

snarkjs v0.7 is the second toolset utilised for building zk-SNARKs after Circom version 2 was released. It is used for the trusted setup (Powers of Tau and circuit-specific phase 2), as well as generating proving keys (.zkey) and generating proofs using Groth16 (.zkey) before exporting the

Solidity verifier contract (iden3, 2023). The groth16.fullProve() function accepts the circuit .wasm, the .zkey, and an input JSON object, and returns the proof and public signals in the format expected by the on-chain verifier.

4.1.4 Frontend Stack

The frontend is a single-page application built without using either a bundler or a build server. This was a deliberate choice to minimise tooling friction for a prototype: the application runs directly in a browser by serving static files.

- React 18.2.0 is loaded from the unpkg CDN as a UMD build (Meta, 2022). React hooks (useState, useEffect, useCallback, useMemo, useRef, useContext) are exposed as global variables on window so that all component files can use them without ES module imports.
- ethers.js v5.7.2 is loaded from cdnjs as a UMD build (Moore, R. (2022)). The v5 API was selected over v6 for its stability and the larger volume of available reference implementations for contract interaction patterns at the time of development.
- QRious 4.0.2 provides QR code generation for token sharing, loaded from cdnjs (Mercer, A. (2017)).
- Babel (@babel/preset-react) transpiles JSX syntax in the component source files to standard JavaScript (Babel Team, 2022). The build step (npm run build) produces a dist/ directory. This avoids requiring a full webpack or Vite pipeline while still allowing JSX authoring.

4.2 Smart Contract Implementation

4.2.1 Gas Optimisation Patterns

All three smart contracts use a consistent set of gas optimisation techniques.

Custom errors. All revert conditions use custom errors declared at the contract level instead of using the require function with a string message. Strings are stored in contract bytecode and cost calldata gas when the error is triggered, while custom errors encode only a 4-byte selector plus any parameters, with no string storage overhead.

```
// Gas-efficient: 4-byte selector, no string storage

error TooManyRequests();

error RequestCooldown(uint256 timeRemaining);


// Used in place of:

// require(_requestCount[requester] < MAX_REQUESTS_PER_DAY, "Too many
requests");

if (_requestCount[requester] >= MAX_REQUESTS_PER_DAY) revert
TooManyRequests();
```

Unchecked arithmetic. Counter increments and loop indices that cannot realistically overflow are wrapped in unchecked blocks to bypass the Solidity 0.8.x overflow checker, saving approximately 80 gas per operation.

```
unchecked {
    tokenId = ++_tokenIdCounter;   // counter cannot reach uint256 max
}


for (uint256 i = 0; i < count; ) {
    // loop body
    unchecked { ++i; }             // i < count guarantees no overflow
}
```

EnumerableSet for pending requesters. The set of addresses with a pending access request for a given token is stored as an EnumerableSet.AddressSet (OpenZeppelin, 2023). This provides O(1) add and remove operations unlike a plain array, which requires a linear scan to remove an element, and supports full enumeration for the paginated getPendingRequests view. When a request is approved or denied, the requester is removed from the set in O(1) without shifting any other elements.

uint64 for timestamps. All block timestamps and durations are stored as uint64 rather than uint256. A uint64 can represent timestamps up to the year 584,542, which is more than sufficient, and fitting two timestamps into a single 32-byte storage slot alongside other fields through slot packing saves one SLOAD and one SSTORE per access.

4.2.2 SoulboundIdentity: ERC-5484 and Non-Transferability

SoulboundIdentity is built on top of an ERC5484 base contract which extends ERC721Enumerable. The non-transferability invariant is enforced at the base contract level by overriding _beforeTokenTransfer to revert unconditionally on any transfer that is not a mint (from the zero address), or an authorised burn.

The one token per address constraint is also enforced at the mint function level.

```
function mint(string calldata ipfsCID, BurnAuth burnAuth) external returns
(uint256) {
    if (balanceOf(msg.sender) > 0) revert AlreadyHasToken();
    return _mint(msg.sender, ipfsCID, burnAuth);
}
```

IPFS CID validation is performed prior to the CID being saved to prevent invalid references from being permanently committed on-chain. The validator accepts both CIDv0 (base58, Qm..., 46 characters long), and CIDv1 (base32, b..., 59 or more characters long), and validates the character set used in either case:

```
function _isValidCID(string memory cid) internal pure returns (bool) {
    bytes memory b = bytes(cid);
    uint256 len = b.length;
    if (len < 46 || len > 100) return false;

    if (b[0] == "Q" && b[1] == "m") {
        if (len != 46) return false;
        // validate base58 character set...
        return true;
    }
    if (b[0] == "b") {
        if (len < 59) return false;
        // validate CIDv1 multibase prefix...
        return true;
    }
    return false;
}
```

4.2.3 SoulboundIdentity: Rate Limiting Implementation

The rate-limiting system uses a day-boundary approach. Each address has a _requestCount (number of requests made that day) and a _lastResetDay (the day index when this request count was last set to zero). The day index is computed as block.timestamp / 1 days, and is therefore deterministic and manipulative-proof because of the limits in QBFT's block production environment.

```
function _checkAndUpdateRateLimit(address requester) internal {
    uint256 currentDay = block.timestamp / 1 days;

    if (currentDay > _lastResetDay[requester]) {
        _requestCount[requester] = 0;
        _lastResetDay[requester] = currentDay;
    }

    if (_requestCount[requester] >= MAX_REQUESTS_PER_DAY) revert
TooManyRequests();

    unchecked { _requestCount[requester]++; }
}
```

The per-token cooldown is checked separately in requestAccess by comparing block.timestamp against _lastRequestTime[msg.sender][tokenId] + REQUEST_COOLDOWN. If the cooldown has not yet elapsed, the revert will include the remaining wait time, allowing the front-end to provide a meaningful countdown rather than a generic error:

```
if (block.timestamp < lastRequest + REQUEST_COOLDOWN) {
    revert RequestCooldown((lastRequest + REQUEST_COOLDOWN) -
block.timestamp);
}
```

4.2.4 CredentialsHub: The Issuer System

CredentialsHub distinguishes two paths for adding credentials. The self-assert path (addCredential, addDegreeCredential, addCertificationCredential) is callable by the token owner and sets verified = false. The issuer path (issueCredential, issueDegreeCredential, issueCertificationCredential) is callable only by addresses in the authorizedIssuers mapping and sets verified = true. The onlyAuthorizedIssuer modifier enforces this at the function level:

```
modifier onlyAuthorizedIssuer(CredentialType credType) {
    if (!authorizedIssuers[credType][msg.sender]) revert
NotAuthorizedIssuer();
    _;
}
```

Issuer authorisation is per credential type, not global. A university authorised to issue Degree credentials cannot issue Certification or WorkExperience credentials under the same authorisation. This granularity prevents authorised issuers from acting beyond their intended scope.

4.2.5 CredentialsHub: Bitmask Access Check

Access checks for credential type queries have been implemented as a single-bitwise expression in the internal helper _canAccessType:

```
function _canAccessType(uint256 tokenId, CredentialType credType)
        internal view returns (bool) {
    if (msg.sender == identity.ownerOf(tokenId)) return true;
    return (grantedTypes[tokenId][msg.sender] >> uint8(credType)) & 1 == 1;
}
```

The token owner always has full access through the first branch. Any other caller will use the credType enum value (0 to 4) as a bit position: by shifting the stored uint8 to the right by credType positions and ANDed with 1 to extract the permission bit. This check costs approximately 200 gas (one SLOAD for grantedTypes plus the bitwise operations) with no branching over credential type cases.

4.2.6 SocialHub: Cached Reputation Totals

A naive implementation of getReputationSummary would iterate over all stored reviews to compute the average score, an O(n) operation that becomes increasingly expensive as review

counts grow. To avoid this issue, SocialHub maintains three cached accumulators that are updated atomically on every submitReview call:

```
// Updated atomically with each review submission:
unchecked {
    _runningScoreTotal[subjectTokenId] += score;
    _reviewCount[subjectTokenId]++;
    if (verified) _verifiedCount[subjectTokenId]++;
}

// O(1) summary read with no iteration:
function getReputationSummary(uint256 tokenId)
        external view returns (ReputationSummary memory) {
    uint256 count = _reviewCount[tokenId];
    if (count == 0) return ReputationSummary(0, 0, 0);
    return ReputationSummary({
        averageScore: _runningScoreTotal[tokenId] / count,
        totalReviews: count,
        verifiedReviews: _verifiedCount[tokenId]
    });
}
```

The cached running total and review count are also the inputs to the reputation_threshold ZKP circuit: the circuit proves that _runningScoreTotal / _reviewCount >= threshold using the cross-multiplication technique described in Chapter 5.

The duplicate-review guard uses a mapping(uint256 => mapping(uint256 => bool)) keyed by (reviewerTokenId, subjectTokenId). The check and set of this mapping operation is O(1) so that no single reviewer can impact a subject's score by submitting numerous reviews.

4.3 Frontend Architecture

4.3.1 Component Structure

The frontend consists of 19 JavaScript files, where each file exposes a single React component as a global variable. The load order of the files in index.html reflects the dependency graph: foundation components (theme.js, toast.js, ui.js) load first; the root app-component.js loads after all utilities; tab and section components load last. The complete component tree is shown in Figure 4.1.

```
AppRoot
  ThemeProvider
    ToastProvider
      App  (app-component.js)
        Header  (header.js)
        Welcome  (welcome.js)          shown before wallet connection
        Navigation  (navigation.js)
        [active tab]
          IdentityTab  (identity-tab.js)
            TokenCard  (token-card.js)
            QRCode  (qr-code.js)
          CredentialsTab  (credentials-tab.js)
          AccessControlTab  (access-control-tab.js)
          SocialTab  (social-tab.js)
            ReputationSection  (reputation-section.js)
            ProjectsSection  (projects-section.js)
            EndorsementsSection  (endorsements-section.js)
          ViewIdentitiesTab  (view-identities-tab.js)
          AnalyticsTab  (analytics-tab.js)
```

4.3.2 Application State

All application-level state is held in the root App component and passed down through props. The principal state variables are listed in Table 4.1.

State Variable	Type	Purpose
`account`	`string | null`	Connected wallet address.
`provider`	`Web3Provider | null`	ethers.js provider wrapping MetaMask.
`signer`	`Signer | null`	ethers.js signer for write transactions.
`contracts`	`object | null`	Initialised contract instances (soulbound, credentials, social).
`userTokens`	`array`	Array of {id, cid} objects for the connected wallet's SBTs.
`selectedToken`	`number | null`	Currently active token ID.
`activeTab`	`string`	Active navigation tab key.
`pendingTxs`	`array`	In-flight transaction tracking objects.

This flat, props-drilling architecture was chosen deliberately over a context-based or Redux-based approach. The application has a single primary flow (connect wallet, select token, interact with credentials) with no complex cross-tree state sharing. The simplicity of direct prop passing outweighs the mild verbosity at this application scale.

The exception is the notification state, which is managed through a ToastProvider context to allow any component at any depth in the tree to trigger user-visible notifications without threading a callback through every intermediate component.

4.3.3 MetaMask Integration and Network Auto-Configuration

Wallet connection is handled by the connectWallet function in App. When a user connects their wallet, the method invokes eth_chainId to obtain the user's current network and then compares that result to CONFIG.CHAIN_ID (424242) to see if it matches. If the networks do not match, it tries to switch automatically:

```
const chainId = await window.ethereum.request({ method: 'eth_chainId' });
if (parseInt(chainId, 16) !== CONFIG.CHAIN_ID) {
    try {
        await window.ethereum.request({
            method: 'wallet_switchEthereumChain',
            params: [{ chainId: `0x${CONFIG.CHAIN_ID.toString(16)}` }],
        });
    } catch (switchError) {
        if (switchError.code === 4902) {
            // Network not known to MetaMask: add it automatically
            await window.ethereum.request({
                method: 'wallet_addEthereumChain',
                params: [{
                    chainId: `0x${CONFIG.CHAIN_ID.toString(16)}`,
                    chainName: CONFIG.NETWORK_NAME,
                    rpcUrls: [CONFIG.RPC_URL],
                }],
            });
        }
    }
}
```

MetaMask uses the ERROR CODE 4902 to indicate an unrecognised chain identifier. When it is caught, wallet_addEthereumChain is called to register the QBFT Besu network in the user's MetaMask with the correct RPC URL, chain name, and chain ID. After this one-time registration, subsequent connections switch directly without requiring the user to add the network again. Therefore, a first-time user will only need two confirmations in order to connect to the application: one to approve the wallet connection and one to confirm the network addition. The network configuration burden is entirely automated.

4.3.4 Resilient Event Log Querying

The application reads historical events (minted tokens, past access requests, credential additions) by querying contract event logs. The Besu RPC node limits the block range of a single eth_getLogs query, which would cause the application to fail silently if a single query were attempted over a large range.

To manage this issue, the queryFilterInChunks function in utils.js divides the block range into chunks of 2,000 blocks and queries each chunk individually. Three failure modes are handled:

1. Block-range limit error (code -32005): This error occurs when a query takes longer than expected to return; it automatically halves the chunk size until reaching a limit of 100 blocks.
2. RPC timeout (over 15 seconds): If an RPC call takes over 15 seconds to return, the application cancels that chunk's request and returns the previously received results without blocking the entire function until it completes.
3. Any other error: All other errors are logged as a warning and the loop exits cleanly, returning whatever events were collected up to that point.

This behavior ensures that the loading state of the user can always be detected, even with slower or partially available RPC nodes instead of having an infinite loop waiting.

4.3.5 Transaction Tracking

All write operations are submitted through the trackTransaction helper, which wraps the transaction receipt wait in a small state machine:

```
const trackTransaction = async (tx, description) => {
    const txId = Date.now();
    setPendingTxs(prev => [...prev, { id: txId, hash: tx.hash, description,
status: 'pending' }]);

    try {
        const receipt = await tx.wait();
        setPendingTxs(prev => prev.map(t =>
            t.id === txId ? { ...t, status: 'confirmed', receipt } : t
        ));
        addToast(`${description} confirmed!`, 'success');
        return receipt;
    } catch (error) {
        setPendingTxs(prev => prev.map(t =>
            t.id === txId ? { ...t, status: 'failed', error: error.message }
: t
        ));
        addToast(`${description} failed: ${error.message}`, 'error');
        throw error;
    }
};
```

The pendingTxs array drives a visible transaction status panel in the header, giving users real-time feedback on transaction confirmation without requiring them to open MetaMask's activity log.

4.3.6 QR Code Integration

Each token has a shareable QR code generated by QRious ((Mercer, A. (2017)) that encodes the token's URL (?token=<tokenId>). When the `QR Code` is scanned by a third party, the application will load the QR Code's token as the initial value/parameter that was passed to the application and set the value for `scannedTokenId in App` to that value to take the user directly into the `view-identity` transaction associated with that token. This offers a convenient means to share information. For example, a professional could show their QR code at a conference, where a recruiter may scan the QR code and immediately see the public identity of the professional as well as be able to submit an access request.

4.4 Deployment

4.4.1 Deployment Sequence

The contracts were deployed to the QBFT Besu network in a strict sequence dictated by their dependency graph:

1. SoulboundIdentity was the first contract deployed as it had no dependencies on any other of the system contracts. The deployment address will be captured to be used in subsequent steps.
2. CredentialsHub was deployed second and received the SoulboundIdentity address as a constructor argument. This address will be referenced immutably.
3. SocialHub was deployed third and also received the SoulboundIdentity address as a constructor argument.
4. The Groth16 verifier contracts are the automatically-generated Solidity verifier which were exported from snarkjs.

The use of the immutable keyword in the Solidity programming language stores constructor argument values in the contract bytecode directly instead of storing them in an additional storage slot. The identity address will be read from Bytecode on each write activity in both CredentialsHub and SocialHub, replacing a storage SLOAD (~2,100 gas) with a bytecode read will result in a gas savings for every credential issue / access granting / review submission for the lifetime of the contract deployment.

4.4.2 Deployed Contract Address Registry

Table 4.2 lists the deployed addresses for the three main contracts on Chain ID 424242.

Contract	Deployed Address
SoulboundIdentity	0xf1FB393025ef07673CcFBD5E83E07f6cF503F8ee
CredentialsHub	0x25154346a75204f80108E73739f0A4AaD4754c8B
SocialHub	0xbe78D2f3c24Abdd91AD8263D7cF10290F94045a7

4.4.3 Network Configuration

The frontend CONFIG objectserves as a repository for all deployment-specific parameters:

```
CONFIG = {
    RPC_URL:      'https://rpc.dimikog.org/rpc/',
    CHAIN_ID:     424242,
    NETWORK_NAME: 'QBFT_Besu_EduNet',
    IPFS_GATEWAY: 'https://ipfs.io/ipfs/',
    CONTRACTS: {
        SOULBOUND_IDENTITY: '0xf1FB393025ef07673CcFBD5E83E07f6cF503F8ee',
        CREDENTIALS_HUB:    '0x25154346a75204f80108E73739f0A4AaD4754c8B',
        SOCIAL_HUB:         '0xbe78D2f3c24Abdd91AD8263D7cF10290F94045a7',
    }
};
```

By storing all the above values in a single object, when the deployment is moved to another network or new contract addresses are used, you only have to modify one file instead of finding an arbitrary number of hardcoded values throughout the component hierarchy.

Chapter 5 - Zero-Knowledge Proof Integration

The details of the Design of the ZKP subsystem are provided in this chapter. In Section 5.1 we will go into how claims of credentials in the real world are mapped to arithmetic circuits. In Section 5.2 we will provide a deeper explanation of the first circuit that was built, the reputation_threshold circuit, which also happens to be the most utilized circuit. The last four circuits and their associated design patterns will be covered in Section 5.3. In Section 5.4 we will go over the Trusted Setup and the way the proving key is generated. In Section 5.5 we will go through the proof creation process in the web browser, and discuss how the ZKP complexity of the proof is hidden from the proof submitter. Finally, Section 5.6 will go through the process of verification by the ZKP on-chain, and what occurs after the proof has been submitted.

5.1 Circuit Design Overview

5.1.1 From Real-World Claim to Arithmetic Circuit

Zero-knowledge proofs operate over arithmetic circuits: directed acyclic graphs of field addition and multiplication gates. A circuit takes a "witness," which is essentially an assignment of values to every wire in the circuit, and checks if they satisfy the gate constraints

of either addition or multiplication, determining whether that witness solves a series of polynomial equations over the prime field (Gennaro et al., 2013).

This translation from natural-language claim to arithmetic constraint is the central design challenge of ZKP engineering. The claim 'my reputation score is above 70' must be re-expressed as a set of field equations that are satisfiable if and only if the claim is true. Table 5.1 shows how each of the five circuits maps a professional credential claim to its circuit constraint.

Circuit	Natural-language claim	Core constraint	Private input	Public input
`reputation_threshold`	Reputation score >= T	`score >= threshold`	`score`	`threshold`
`gpa_threshold`	GPA >= T (x100 scaled)	`gpa >= threshold`	`gpa`	`threshold`
`work_experience_threshold`	Total experience >= T years	`totalYears >= threshold`	`totalYears`	`threshold`
`degree_category`	Degree is in category C	`category == claimedCategory`	`category`	`claimedCategory`
`cert_domain`	Certification is in domain D	`domain == claimedDomain`	`domain`	`claimedDomain`

5.1.2 Private vs. Public Inputs

Each circuit within the system establishes a distinct separation of boundaries between public & private inputs.

The private input (the witness) is only known to the prover. It contains sensitive credential values (the full score, the GPA, the time a person worked, or the category enum) that the user doesn't want disclosed. The Groth16 proof generating system (Groth, 2016) uses private inputs as part of its proof object computation process. The proof generated cannot be used to extract any portion of the private inputs via verification, based on its inherent mathematical properties.

The public input is known by both the prover & verifier. These credentials provide the information required to validate the claim being verified, using the threshold or category of the claim. These public inputs are sent along with the proof generated on-chain to the on-chain verifier; these public inputs are contained in the emitted events and can be viewed by anyone who queries the blockchain for that proof.

The result is that a verifier learns exactly what they requested, for example 'this credential is above threshold T' or 'this degree is in category C', and nothing more.

5.1.3 Prototype Scope

The circuits implemented in this thesis are deliberately minimal proof-of-concept circuits. Each proves a single claim with the minimum number of constraints necessary to demonstrate the

end-to-end ZKP flow: proof generation, on-chain verification, and user-transparent interaction. These circuits are in accordance with the overall purpose of the project, which is to show the full operation of the system end-to-end, rather than to develop fully production quality circuits with complete data binding to the on-chain state.

Section 5.2.5 explains what a production circuit would require to use the reputation threshold and the reasons for selecting a simpler formulation in the prototype.

5.2 The reputation_threshold Circuit

5.2.1 What the Circuit Proves

The reputation_threshold circuit proves the claim: the reputation score associated with this identity is greater than or equal to the threshold T.

The circuit takes two inputs:

1.Private: score, the actual reputation score of the token holder, computed from the cached _runningScoreTotal / _reviewCount values in SocialHub.

2.Public: threshold, the minimum score the verifier requires, set at proof-generation time.

The primary constraint enforced by the circuit is score >= threshold. If this condition is met, the prover will be able to generate a valid Groth16 proof. The verifier can verify that the prover knows a score satisfying the threshold constraint by verifying the proof provided and the public threshold, thereby not being permitted to see the actual score.

5.2.2 Circuit File Artefacts

The Circom 2 compiler (iden3, 2022) creates two artifacts for use when generating a proof:

- reputation_threshold.wasm (37 KB): This is a WebAssembly witness generator. Given the circuit inputs as a JSON object, the WASM module computes the full witness: all intermediate signal values required to satisfy the R1CS. The witness generator will run entirely within your browser without any servers being involved whatsoever.
- reputation_threshold_final.zkey (22 KB): This is a "proving key" produced by the circuit-specific phase 2 of the trusted setup. This encodes the structured reference string required for Groth16 proving. It is loaded alongside the WASM file at proof-generation time.

The relatively small file sizes reflect the simplicity of the prototype circuit: a small number of R1CS constraints leads to a compact SRS and a small WASM binary. Both files are served as static assets and are cacheable after the first load.

5.2.3 Proof Generation Call

The proof is generated by a single snarkjs call in the frontend (iden3, 2023):

```
const { proof, publicSignals } = await window.snarkjs.groth16.fullProve(
    { score: score.toString(), threshold: threshold.toString() },
    'circuits/reputation_threshold.wasm',
    'circuits/reputation_threshold_final.zkey'
);
```

groth16.fullProve() performs two steps internally: it runs the WASM witness generator to compute the full witness based on its inputs, and then it runs the Groth16 prover on the witness and proving key to produce the proof. The proof that groth16.fullProve() returns consists of three BN128 elliptic curve points (pi_a, pi_b, pi_c) and publicSignals include the circuit's public output values.

5.2.4 Proof Format and the pB Coordinate Swap

The Groth16 proof consists of three curve points over the BN128 pairing-friendly elliptic curve (Groth, 2016):

- pi_a — a point on G1, represented as two 32-byte field elements [x, y]
- pi_b — a point on G2, represented as two pairs of 32-byte field elements [[x0, x1], [y0, y1]]
- pi_c — a point on G1, represented as two 32-byte field elements [x, y]

The auto-generated Solidity verifier contract expects the G2 point in the opposite coordinate order from what snarkjs returns. This coordinate swap must be applied before submitting the proof on-chain:

```
const pA = [proof.pi_a[0], proof.pi_a[1]];
// G2 coordinates are swapped: [1][0] before [0][0], [1][1] before [0][1]
const pB = [
    [proof.pi_b[0][1], proof.pi_b[0][0]],
    [proof.pi_b[1][1], proof.pi_b[1][0]]
];
const pC = [proof.pi_c[0], proof.pi_c[1]];
```

This swap is well known and documented requirement of the snarkjs and Solidity verifier interface, arising from the different affine coordinate conventions between snarkjs's output, and what is expected by the EVM's BN128 precompile for encoding a G2 point. All circuits require the same swap.

5.2.5 What a Production Circuit Would Add

The prototype circuit is able to verify the claim based on a score value directly from the frontend. For the production implementation of this same claim, there would need to be three additional extensions to the original claim:

1. On-chain data binding. In its current state, the front-end gets the average score from the cache on the SocialHub and forwards it as a private input before it is submitted. A malicious prover could supply any value rather than the actual on-chain score. A production circuit would include the individual review scores as private inputs, recompute the average inside the circuit, and include a Poseidon commitment (Grassi et al., 2021) to the score array as a public input, allowing the verifier contract to confirm the proof was generated from the actual on-chain score state.
2. Range checking. The prototype does not check that the average score is in the valid boundary of [0,100]. In a production circuit, the average score will be bounded using a bit decomposition gadget in order to restrict the prover from being able to enter an out-of-bounds value that satisfies the threshold condition.
3. tokenId binding. The prototype proof does not reference which token it was generated for. In a production circuit, the tokenId will be included as a public input, which our on-chain verifier can then use to ensure that the proof was created for the correct identity.

These extensions represent a natural evolution path from prototype to production, as we will discuss in more detail in Chapter 7.

5.3 The Remaining Four Circuits

5.3.1 gpa_threshold

Claim proved: The GPA (Grade Point Average) of the token will be greater than or equal to the "threshold" T (both are expressed as integers and are each scaled by a factor of 100) for the purposes of comparison.

- Private: gpa, the GPA value stored in CredentialsHub.degreeGPA[credentialId], encoded as an integer multiplied by 100 (for example, 350 represents GPA 3.50).
- Public: threshold, the minimum GPA required, also multiplied by 100 (for example, 300 represents GPA 3.00).

Core constraint: gpa >= threshold. The purpose of storing the GPA in scaled integer form directly on-chain is to keep two decimal fraction precision without the use of floating-point arithmetic (which does not exist in Solidity or in an arithmetic circuit). Both the confidential and public inputs will use the same scaling factor, so therefore the comparison will be conducted as exact integer mathematics without precision loss.

Circuit artefacts: gpa_threshold.wasm (37 KB) and gpa_threshold_final.zkey (22 KB), matching the reputation_threshold file sizes and reflecting a structurally identical circuit with different semantic input names.

Frontend call:

```
await window.snarkjs.groth16.fullProve(
    { gpa: gpa.toString(), threshold: threshold.toString() },
    'circuits/gpa_threshold.wasm',
    'circuits/gpa_threshold_final.zkey'
);
```

5.3.2 work_experience_threshold

Claim proved: the total accumulated work experience is at least T years.

- Private: totalYears, the number of complete years of work experience computed by CredentialsHub.getTotalWorkExperience().
- Public: threshold, the minimum number of years required.

Core constraint: totalYears >= threshold. The totalYears vs. threshold comparison is performed as a non-strict inequality in recognition of how work experience thresholds are conventionally expressed in professional contexts: "at least R3 years" (not "greater than R3 years") when affirming that an individual possesses sufficient qualifications.The underlying circuit constraint is totalYears - threshold >= 0.

The on-chain work experience total is computed by getTotalWorkExperience(), which sums the duration of all non-revoked WorkExperience credentials using issueDate as the start date and expiryDate as the end date, substituting block.timestamp for positions still marked as active, and returns whole years and remaining months. The circuit uses only the whole-year total.

Circuit artifacts: work_experience_threshold.wasm (37 KB) and work_experience_threshold_final.zkey (22 KB).

5.3.3 degree_category

Claim proved: This token has an active degree credential for category C.

- Private: category, the DegreeCategory enum value of an active degree credential, ranging from 0 to 5.
- Public: claimedCategory, the category the prover asserts membership in, also in the range 0 to 5.

Core constraint: category == claimedCategory. This circuit is building a proof of equality and not inequality. The constraint is enforced as category - claimedCategory == 0.

This circuit is structurally different from the threshold circuits, and that difference is visible in the file sizes: degree_category.wasm (35 KB) and degree_category_final.zkey (3.5 KB). The proving key is about six times smaller than the proving keys for the inequalty proof circuits; the equality proof requires fewer R1CS prover constraints than those required for the range-checked inequalty constraints that will require use of bit decomposition gadgets. Therefore, there is now a smaller SSR produced that can generate a smaller .zkey.

The frontend performs a pre-condition check before invoking the prover, confirming that an active degree credential with the claimed category exists in the viewed token's credential set. This prevents any proof generation failure should no active degree credential be found that matched.

5.3.4 cert_domain

Claim proved: The certification credential has an active status in the certification domain D.

- Private: domain, the CertificationDomain enum value of an active certification credential, ranging from 0 to 4.
- Public: claimedDomain, the domain being asserted.

Core constraint: domain == claimedDomain. The circuit is structurally identical to degree_category but operates over the CertificationDomain enum (five values: ITSoftwareDevelopment, BusinessManagement, HealthMedicine, DataAIAnalytics, CommunicationMarketing) instead of DegreeCategory enumeration (with six values). Circuit artefacts: cert_domain.wasm (35 KB) and cert_domain_final.zkey (3.5 KB), matching degree_category and confirming equivalent circuit structure and constraint count.

5.3.5 Circuit Comparison Summary

Circuit	.wasm size	.zkey size	Constraint type	Inputs
`reputation_threshold`	37 KB	22 KB	Non-strict inequality	`score, threshold`
`gpa_threshold`	37 KB	22 KB	Non-strict inequality	`gpa, threshold`
`work_experience_threshold`	37 KB	22 KB	Non-strict inequality	`totalYears, threshold`
`degree_category`	35 KB	3.5 KB	Equality	`category, claimedCategory`
`cert_domain`	35 KB	3.5 KB	Equality	`domain, claimedDomain`

The .zkey size is the most informative complexity indicator. The 22 KB proving keys for the three inequality circuits reflect a substantially larger R1CS than the 3.5 KB keys for the two equality circuits. Inequality constraints in arithmetic circuits require a range check, which is accomplished through a bit decomposition of the difference value to ensure that it is not

negative. This decomposition adds significantly more constraints than a simple equality check, producing the larger SRS and correspondingly larger proving key.

5.4 Trusted Setup and Proving Key Generation

5.4.1 The Two-Phase Trusted Setup

The Groth16 Protocol requires the use of a trusted set up ceremony that is performed once to create a Structured Reference String (SRS) from which the Proving Key and the Verification Key are derived (Groth, 2016). The Trusted Set up ceremony consists of two stages.

Phase 1: Powers of Tau. The first phase generates a universal SRS that is circuit-independent. It is called Powers of Tau because the SRS consists of powers of a secret scalar evaluated on the BN128 elliptic curve. Anyone who knows the secret can construct a proof for any circuit in existence. Therefore, the secret must be securely destroyed or disposed of in accordance with the established and accepted cryptographic methodologies. This is achieved through a multi-party computation ceremony in which each participant contributes fresh randomness and proves they deleted their secret contribution. If any single participant acted honestly, the SRS is secure.

For this project, the ceremony file that was generated by the publicly available Hermez Network Powers of Tau Ceremony (powersOfTau28_hez_final_12.ptau) will be used. This file supports circuits with an upper limit of 4,096 R1CS constraints (that is, 2^12=4,096 R1CS constraints) (Hermez Network, 2020). The ceremony incorporated 55 independent participant contributions, making collusion computationally implausible.

Phase 2: Circuit-specific contribution. The second phase is to take the SRS from phase 1 and specialize it for a specific circuit's RC1CS structure as such to create the proving key (.zkey) and the verification key that will be embedded in the exported Solidity verifier contract. Phase 2 also requires a secret contribution that must be discarded, but it is circuit-specific and can be performed by the deployer alone. For a prototype, a single-participant phase 2 is acceptable. However, a production deployment would require a multi-party phase 2 ceremony to eliminate the single-party trust assumption.

5.4.2 Key Generation Sequence

The key generation sequence for each circuit follows these snarkjs commands:

```
# 1. Compile the Circom circuit to R1CS and WASM

circom circuit.circom --r1cs --wasm --sym


# 2. Phase 2 setup: specialise the SRS to this circuit

snarkjs groth16 setup circuit.r1cs powersOfTau28_hez_final_12.ptau
circuit_0000.zkey


# 3. Contribute randomness to phase 2 (secret discarded after)

snarkjs zkey contribute circuit_0000.zkey circuit_final.zkey --
name="contributor"


# 4. Export the Solidity verifier contract

snarkjs zkey export solidityverifier circuit_final.zkey Verifier.sol


# 5. Export the verification key for optional local verification

snarkjs zkey export verificationkey circuit_final.zkey verification_key.json
```

The resulting circuit_final.zkey file is the proving key served to the browser. The Verifier.sol program is now active on the blockchain for verification purposes. The verification_key.json file allows for local verification without an on-chain call.

5.4.3 Trusted Setup Security Assumptions

The security of the system rests on the assumption that at least one participant in the Hermez Powers of Tau ceremony honestly deleted their contribution secret. Due to the scale of this ceremony, it is generally accepted that this assumption has been fulfilled. One of the contributions made during the circuit-specific phase 2 of the ceremony is made by the project deployer, which creates an assumption of single-party trust on a weaker basis for the prototype project. However, in the production environment for the phase 2 contribution will be made with multiple party trust by using the snarkjs zkey beacon command with a verifiable random beacon to eliminate the single-party trust assumption for the production deployment.

5.5 In-Browser Proof Generation

5.5.1 The Full Proof Generation Flow

When a user initiates a ZKP attestation from the Access Control tab, the following sequence executes entirely in the browser:

```
User clicks Verify
  |
  v
Frontend reads on-chain data
(score from SocialHub / GPA from CredentialsHub / etc.)
  |
  v
Frontend checks pre-condition
(score >= threshold? matching credential exists?)
  | Yes
  v
snarkjs.groth16.fullProve(inputs, .wasm path, .zkey path)
  |
  |-- WASM witness generator runs
  |   (computes all intermediate signals from inputs)
  |
  +-- Groth16 prover runs
      (produces proof: pi_a, pi_b, pi_c + publicSignals)
  |
  v
Frontend applies pB coordinate swap
  |
  v
verifier.verifyProof(pA, pB, pC, publicSignals)
(static call to on-chain verifier, no gas consumed)
  |
  v
Display result to user (Verified on-chain)
```

The entire process of generating an attestation from the time the user clicked until receiving verification results (on-chain) is completed within the user's browser without any input or feedback from the user other than screen messages that transition through three states: generating ('Generating proof...'), verifying ('Verifying on-chain...'), and then either success or error.

5.5.2 State Machine in the Frontend

Each of the five circuits has a dedicated React state pair in AccessControlTab: a *ZkpStatus string (idle, generating, verifying, success, or error) and a *ZkpMessage string. These are updated synchronously as the proof flow progresses:

```
setRepZkpStatus('generating');
setRepZkpMessage('Generating proof...');
// ... await snarkjs call ...
setRepZkpStatus('verifying');
setRepZkpMessage('Verifying on-chain...');
// ... await verifier call ...
setRepZkpStatus('success');
setRepZkpMessage(`Reputation >= ${threshold} -- verified on-chain.`);
```

If snarkjs throws (for example, if the inputs are invalid or the circuit constraint is not satisfied), the catch block sets the status to 'error' with the error message. If the on-chain verifier returns false (a valid proof format but incorrect for the verification key), the error state is set explicitly. In all cases the user is given a non-technical error message and has no exposure to the underlying cryptographic error condition.

5.5.3 Pre-Condition Checking

Before invoking fullProve, the frontend performs a pre-condition check: if the private input already fails the claim (for example, if the score is not greater than or equal to the threshold), the proof attempt is short-circuited and an error state is set immediately. Therefore, the frontend saves time and avoids waiting several seconds to generate a proof that will be proved invalid:

```
if (score < threshold) {
    setRepZkpStatus('error');
    setRepZkpMessage(`Score is NOT above ${threshold}.`);
    return;
}
```

The reason for including this pre-condition check is not because it adds additional security. The circuit itself includes constraints that will reject any invalid proof anyway. Rather, the pre-condition check is a usability feature because it improves the user experience by not having to wait several seconds for the proof to finish and then return an error message.

5.5.4 snarkjs as a Browser Dependency

snarkjs is loaded into the browser via a script tag that exposes window.snarkjs. Each handler function checks for its presence before attempting any proof operation:

```
if (!window.snarkjs) {
    showNotification('snarkjs not loaded', 'error');
    return;
}
```

This serves as a guard so that silent failure does not occur in cases where the script does not successfully load due to network issues, and it gives the user a clear error message about the problem.

5.6 On-Chain Verification

5.6.1 The Auto-Generated Groth16 Verifier

Each circuit has its own Solidity verifier contract exported by snarkjs (iden3, 2023) with all five verifiers exposing the same interface:

```
function verifyProof(
    uint[2] calldata _pA,
    uint[2][2] calldata _pB,
    uint[2] calldata _pC,
    uint[1] calldata _pubSignals
) public view returns (bool)
```

The verifier performs the Groth16 verification equation: a series of BN128 pairing operations using the EVM's ecAdd, ecMul, and ecPairing precompiles (Reitwiessner, 2017; Buterin & Reitwiessner, 2017). The equation checks that the proof is consistent with the public signals under the verification key embedded in the contract bytecode. The function is marked as view, meaning it reads no state and costs approximately 280,000 gas when called as part of a transaction (exact cost subject to the precompile pricing defined in EIP-1108, Buterin & Skordis, 2019), or zero gas when called as a static read-only query against a local node.

5.6.2 Deployed Verifier Contracts

Each circuit has their own deployed verifier contract on Chain ID 424242. The contracts are accessed from the frontend via dedicated ABI entries in contracts.js.

The ABI constant and associated configuration key values for each of the five verifiers are listed in Table 5.3.

Circuit	ABI constant	Config key
reputation_threshold	GROTH16_VERIFIER_ABI	CONTRACTS.REPUTATION_VERIFIER
gpa_threshold	GPA_VERIFIER_ABI	CONTRACTS.GPA_VERIFIER
degree_category	DEGREE_CATEGORY_VERIFIER_ABI	CONTRACTS.DEGREE_CATEGORY_VERIFIER
cert_domain	CERT_DOMAIN_VERIFIER_ABI	CONTRACTS.CERT_DOMAIN_VERIFIER
work_experience_threshold	WORK_EXPERIENCE_VERIFIER_ABI	CONTRACTS.WORK_EXPERIENCE_VERIFIER

All five verifiers share the same ABI signature (a verifyProof function taking three curve points and one public signal array), as generated by snarkjs for single-public-signal Groth16 circuits.

5.6.3 The Verification Call

The frontend does not submit a blockchain transaction for verification. Instead, it makes a static call to the verifier contract. In ethers.js v5, calling a view function executes locally on the connected node without broadcasting a transaction:

```
const provider = new ethers.providers.JsonRpcProvider(CONFIG.RPC_URL);
const verifier = new ethers.Contract(
    CONFIG.CONTRACTS.REPUTATION_VERIFIER,
    GROTH16_VERIFIER_ABI,
    provider  // provider, not signer: read-only call, no transaction sent
);
const isValid = await verifier.verifyProof(pA, pB, pC, publicSignals);
```

Thus, passing the provider rather than the signer to the contract constructor ensures no transaction is sent and no gas is consumed for the verification call itself. Meaning that everyone is able to verify a proof without having ether.

5.6.4 Gas Cost of On-Chain Verification

The EVM precompile operations invoked by the Groth16 verifier have fixed gas costs defined in EIP-196 and EIP-197 (Reitwiessner, 2017; Buterin & Reitwiessner, 2017):

Operation	Gas cost
ecAdd (G1 point addition)	150 gas
ecMul (G1 scalar multiplication)	6,000 gas
ecPairing (base cost)	45,000 gas
ecPairing (per pairing operation)	34,000 gas

A Groth16 verifier requires three pairing operations (each consuming one G2 and one G1 point). The total precompile cost is approximately 45,000 + (3 x 34,000) = 147,000 gas for the pairings, plus approximately 133,000 gas for calldata decoding, memory operations, and surrounding EVM execution, totalling approximately 280,000 gas for a full verifyProof transaction.

On the QBFT Besu network, the gas price is adjustable and is typically set to zero for permissioned networks, making verification economically free. On a public Ethereum mainnet deployment at 20 gwei per gas, the verification cost would be approximately 0.0056 ETH per proof. Which is a feasible and reasonable cost for a high-value professional attestation that replaces an entire background check process.

Chapter 6 - Evaluation

In this chapter, SoulBound Identity is evaluated under four separate perspectives: functional correctness (Section 6.1), gas cost efficiency (Section 6.2), ZKP subsystem performance (Section 6.3), and security properties (Section 6.4). Section 6.5 describes the known limitations of the full-functioning prototype. Section 6.6 provides structured answers to the three formal research questions stated in Chapter 1.

6.1 Functional Evaluation

The system was evaluated against five end-to-end user journeys. Each journey was executed on the deployed QBFT Besu network (Chain ID 424242) using two separate MetaMask wallet accounts to test multi-party interactions.

6.1.1 User Journey 1: Identity Creation and Sharing

Step	Expected Behaviour	Result
Open application in browser	Welcome screen displayed; wallet connection prompt shown.	Pass
Click Connect Wallet	MetaMask prompts for account access; on an unknown network, wallet_addEthereumChain auto-registers QBFT_Besu_EduNet.	Pass
Mint new soulbound token	Transaction submitted; token minted to wallet address; token card appears in grid.	Pass
Second mint attempt (same wallet)	AlreadyHasToken() revert; user notified.	Pass
Generate QR code	QR encodes ?token=<id> URL; scanning loads view-identity flow for that token.	Pass
Attempt transfer via wallet	ERC-5484 override reverts the transfer unconditionally.	Pass

6.1.2 User Journey 2: Credentials Management

Step	Expected Behaviour	Result
Add self-asserted degree (with GPA)	Credential stored with verified = false; GPA stored x100; DegreeCategory enum set.	Pass
Add self-asserted certification	Credential stored with verified = false; CertificationDomain enum set.	Pass
Add work experience (start/end dates)	Credential stored; getTotalWorkExperience() returns accumulated duration.	Pass
Add skill credential	Credential stored with SkillCategory.	Pass
Revoke a credential	Status transitions to Revoked; credential excluded from active queries.	Pass
Credential summary cards	Counts update correctly per type; owner sees all; third party sees only granted types.	Pass
Attempt to add more than 100 credentials of one type	TooManyCredentials() revert enforced at MAX_CREDENTIALS_PER_TYPE = 100.	Pass

6.1.3 User Journey 3: Access Control

Step	Expected Behaviour	Result
Account B requests access to Account A's token	AccessRequested event emitted; request appears in A's pending list.	Pass
Account A approves with 30-day duration	AccessApproved event emitted; Account B can call canView() and receives true.	Pass
Account A denies a second request	AccessDenied event emitted; Account B cannot view.	Pass
Account B requests again within 1 hour	RequestCooldown(timeRemaining) revert.	Pass
Account B makes 10 requests in one day	Eleventh request triggers TooManyRequests() revert.	Pass
Account A grants bitmask 0b00101 to Account B	Account B can access Degree (bit 0) and WorkExperience (bit 2); Certification query returns NoAccess().	Pass
Access expires after duration elapses	canView() returns false post-expiry without requiring any transaction.	Pass

6.1.4 User Journey 4: Social Features

Step	Expected Behaviour	Result
Account B submits review for Account A	Score stored; _runningScoreTotal and _reviewCount updated atomically.	Pass
Attempt self-review	CannotReviewSelf() revert.	Pass
Account B attempts second review for same subject	AlreadyReviewed() revert.	Pass
getReputationSummary()	Returns O(1) average from cached totals; result consistent with manual calculation.	Pass
Endorse standard skill (e.g. BlockchainDevelopment)	_standardSkillCount incremented; duplicate endorsement from same endorser rejected.	Pass
Create project with IPFS metadata hash	Project stored; status initially Planning; updateProjectStatus advances to Completed.	Pass

6.1.5 User Journey 5: ZKP Verification

All five ZKP circuits were tested end-to-end: proof generation in-browser followed by on-chain verification through the deployed verifier contracts.

Circuit	Pre-condition check	Proof generation	On-chain verification
`reputation_threshold`	Score below threshold: immediate error, no proof attempted.	Valid proof generated for score at or above threshold.	verifyProof() returns true.
`gpa_threshold`	GPA below threshold: immediate error.	Valid proof generated for GPA at or above threshold.	verifyProof() returns true.
`work_experience_threshold`	Years below threshold: immediate error.	Valid proof generated for years at or above threshold.	verifyProof() returns true.
`degree_category`	No matching active credential: immediate error.	Valid proof generated when credential exists.	verifyProof() returns true.
`cert_domain`	No matching active credential: immediate error.	Valid proof generated when credential exists.	verifyProof() returns true.

The end-to-end connections of each of the five circuits have been successfully verified. All proof verifications returned false when using a proof created for one circuit submitted for verification against a different circuit's verification process. This confirms that cross-circuit reuse of proofs will be rejected.3

6.2 Gas Cost Analysis

6.2.1 Measured Operation Costs

Gas costs were measured against the deployed contracts on the QBFT Besu network (Hyperledger, 2022). Table 6.6 reports the gas consumed per operation.

Operation	Contract	Gas used	ETH at 20 Gwei
Mint soulbound token	`SoulboundIdentity`	~200,000	0.0040 ETH
Add self-asserted credential	`CredentialsHub`	~150,000	0.0030 ETH
Issue verified credential (authorised issuer)	`CredentialsHub`	~160,000	0.0032 ETH
Add degree with GPA	`CredentialsHub`	~165,000	0.0033 ETH
Request access	`SoulboundIdentity`	~80,000	0.0016 ETH
Approve access	`SoulboundIdentity`	~90,000	0.0018 ETH
Deny access	`SoulboundIdentity`	~70,000	0.0014 ETH
Revoke credential	`CredentialsHub`	~60,000	0.0012 ETH
Submit peer review	`SocialHub`	~120,000	0.0024 ETH
Create project	`SocialHub`	~130,000	0.0026 ETH
Endorse standard skill	`SocialHub`	~100,000	0.0020 ETH
Submit ZKP proof (on-chain record)	`ReputationProofRegistry`	~280,000	0.0056 ETH

ETH costs are indicative at 20 Gwei per gas, used as a public Ethereum mainnet reference point. The QBFT Besu network uses a configurable gas price that is typically set to zero or a negligible nominal value on a private consortium network.

6.2.2 Cost Drivers

The mint operation (approximately, 200K gas) is the most expensive single operation when done as part of the routine identity operation. Its primary cost components are: two SSTORE operations for token ownership and metadata CID (~40K gas), one SSTORE for the mint timestamp (~20K gas), enumerable token set updates (~50K gas), and ERC-721 event emission with calldata (~30K gas). The remaining cost represents the contract dispatch overhead, as well as the CID validation loop.

The ZKP proof submission (approximately, 280K gas) is also the most expensive single operation overall. This cost is dominated by the BN128 pairing precompile calls: the base cost of ecPairing is 45,000 gas plus 34,000 gas per pair, totalling approximately 147K gas for the three pairings required by Groth16 (Buterin & Skordis, 2019). The remainder of the cost is associated with decoding calldata and performing an SSTORE update to the registry.

6.2.3 Comparison to Naive Alternatives

Naive full-data storage. A system that stored all credential fields on-chain would require approximately 5 to 10 additional SSTORE operations per credential, adding roughly 100,000 to 200,000 gas per credential addition. Against the current on-chain-only cost of approximately 150,000 gas, a naive full-storage design would cost 250,000 to 350,000 gas per credential. The IPFS off-chain split reduces per-credential cost by approximately 40 to 60 percent.

Naive per-grant access control. A per-grant mapping (one SSTORE per credential type per viewer) would require five separate storage writes to grant access to all credential types, approximately 100,000 gas. The bitmask scheme (one uint8 SSTORE) costs approximately 20,000 gas for a new entry, an 80 percent reduction.

No rate limiting. Without the rate-limiting overhead (~5,000 gas per request for the two mapping reads and one write), requestAccess would cost approximately 75,000 gas rather than ~80,000. The 5,000 gas premium for spam protection is negligible relative to the protection it provides.

6.3 ZKP Subsystem Performance

6.3.1 Circuit Artefact Sizes

Table 6.7 reports the exact file sizes measured from the deployed circuit artefacts.

Circuit	.wasm (bytes)	.wasm (KB)	.zkey (bytes)	.zkey (KB)	Total (KB)
reputation_threshold	37,105	36.2	21,658	21.2	57.4
gpa_threshold	37,084	36.2	21,658	21.2	57.4
work_experience_threshold	37,143	36.3	21,974	21.5	57.8
degree_category	35,823	35.0	3,562	3.5	38.5
cert_domain	35,811	35.0	3,562	3.5	38.5
Total (all 5 circuits)	**182,966**	**178.7**	**72,414**	**70.7**	**249.4**

The total download footprint of all five circuits is about 249 KB, which is similar to downloading one medium-resolution image. Under HTTP caching these files are downloaded once and served from the browser cache on subsequent visits. The disparity between .zkey sizes (21 KB versus 3.5 KB) reflects the greater R1CS constraint count of the inequality circuits, which require a bit decomposition for range checking, as discussed in Section 5.3.5.

6.3.2 In-Browser Proof Generation Timing

The specific time it takes to generate a proof will be dependent on the hardware of the device, the JavaScript engine used and the amount of load currently placed on the operating system. Because dedicated hardware benchmarking was beyond the scope of this prototype evaluation, the figures in Table 6.8 are estimated based on the circuit artefact sizes and typical snarkjs WASM performance profiles reported in the literature (iden3, 2023), rather than measured directly on the deployment hardware.

Device class	Example hardware	Estimated generation time
Modern desktop or laptop	Intel Core i7 / Apple M2	0.5 to 2 seconds
Mid-range laptop	Intel Core i5 (2019 to 2021)	2 to 5 seconds
Low-end or older device	Intel Core i3 / older Atom	5 to 15 seconds
Modern smartphone	iPhone 14 / Pixel 7	3 to 8 seconds
Low-end smartphone	Budget Android (2 to 3 GB RAM)	10 to 30 seconds, or failure

The equality circuits (degree_category and cert_domain) generate proofs significantly faster than the inequality circuits due to their smaller .zkey files (3.5 KB versus 21 to 22 KB), because they have fewer constraints evaluated during proving.

6.3.3 Proof Object Size

A Groth16 proof over BN128 consists of three curve points: two G1 points (each 64 bytes uncompressed, representing two 32-byte coordinates) and one G2 point (128 bytes, representing four 32-byte coordinates). The total proof object size is 192 bytes, regardless of circuit complexity. Because the total size of a proof is based solely on the characteristics of the proof system, this amount will be the same across all five circuits (Groth, 2016).

The publicSignals array for all five circuits contains a single element (one uint256), adding 32 bytes. The total on-chain calldata for a verifyProof call is therefore approximately 224 bytes plus ABI encoding overhead of roughly 32 bytes for the selector and length fields, totalling approximately 256 bytes.

6.4 Security Analysis

6.4.1 Access Control Correctness

The bitmask access control system in CredentialsHub has a formally simple invariant: a caller can access credential type T for a given token if and only if (grantedTypes[tokenId][caller] >> uint8(T)) & 1 == 1, or caller == identity.ownerOf(tokenId). This invariant is enforced at the contract level on every read function (getCredential, getCredentialsByType, getActiveCredentials, getCredentialSummary) without exception. There is no execution path through the contract that returns credential data to an unauthorised caller.

The analogous invariant holds in SocialHub for the three section bits. The owner-always-has-access clause relies on a pure read of identity.ownerOf(tokenId), a call to the trusted SoulboundIdentity contract rather than a local state variable, which means it cannot be manipulated by the caller.

A potential concern is cross-contract re-entrancy: a malicious contract calling getCredential during a callback from an earlier contract call. All write functions in CredentialsHub and SocialHub that modify state are protected by OpenZeppelin's ReentrancyGuard (OpenZeppelin, 2023), which prevents re-entrant calls. The read-only view functions do not modify state and are not re-entrancy risks by construction.

6.4.2 Non-Transferability Enforcement

The ERC-5484 non-transferability invariant is enforced at the base contract level by overriding the ERC-721 _beforeTokenTransfer hook to revert on any transfer that is not a mint or an authorised burn (Cai, 2022). This means the invariant cannot be circumvented by calling any standard ERC-721 function, including transferFrom, safeTransferFrom, or approve, on the SoulboundIdentity contract. The only way to move a token is to burn it and re-mint, which requires the appropriate burnAuth level and destroys the token's entire history.

The one-token-per-address constraint, enforced by if (balanceOf(msg.sender) > 0) revert AlreadyHasToken() in mint(), prevents more than one identity token being held by any given address, thus maintaining the identity fidelity characteristic of identity tokens.

6.4.3 Rate Limiting Effectiveness

The two-tier rate-limiting system (a one-hour per-token cooldown and a ten-requests-per-day global limit) provides meaningful protection against spam access requests while imposing negligible friction on legitimate users.

An attacker attempting to overwhelm a token owner with access requests is limited to at most 10 requests per day across all target tokens and cannot re-request the same token more than once per hour. The gas cost of each request (~80,000 gas) imposes an economic barrier: at 20 Gwei, 10 requests cost 0.016 ETH. On a public mainnet, the cost of a sustained spam campaign is economically significant relative to any expected benefit.

The rate limit counters reset based on block.timestamp / 1 days. On a QBFT network with deterministic finality and honest validators, this boundary is reliable. On a public Proof-of-Work or Proof-of-Stake network, minor timestamp manipulation by block proposers within the approximately 15-second tolerance could shift the reset boundary slightly, but would not be able to meaningfully defeat the daily limit.

6.4.4 ZKP Soundness Guarantees

The Groth16 proof system provides computational soundness under the q-Strong Diffie-Hellman (q-SDH) assumption over the BN128 curve (Groth, 2016; Boneh & Boyen, 2004). This means three things in practice.

First, a prover without a valid witness (a score value genuinely satisfying the threshold) cannot produce a proof that passes verifyProof() except with probability negligible in the security parameter. No known polynomial-time algorithm can produce a valid Groth16 proof for a false statement without knowing the toxic waste from the trusted setup.

Second, the BN128 curve is well-studied and its discrete logarithm problem is currently believed to offer approximately 128 bits of security against the best known algorithms.

Third, the verifyProof function in the auto-generated Solidity verifier performs the complete Groth16 verification equation using the EVM's native pairing precompile, which has been audited as part of the Ethereum protocol specification. There is no implementation gap between the mathematical specification and the on-chain check.

The soundness of the proof is based on the correctness of the trusted setup. If the parties involved in the Hermez ceremony were to act collusively to keep the toxic waste, then they could generate valid proof for any given circuit using that particular SRS. The single-party phase

2 contribution in the prototype introduces a single point of trust for circuit-specific soundness. Both limitations are acknowledged in Section 6.5.

6.4.5 Replay Attack Resistance

In the prototype implementation, the ZKP circuits do not include a tokenId or data commitment as a public input. This means a valid proof generated for one token's score being above threshold T is technically reusable for any other token: the verifier contract does not check that the proof was generated for the specific token submitting it. The system mitigates this at the application layer by requiring an explicit tokenId to be provided when submitting a proof. A submitter who presents a valid proof alongside an incorrect tokenId could therefore produce a misleading on-chain record.

However, the prototype does not cryptographically enforce this binding at the circuit level. A production-ready implementation would bind proofs to specific tokens by incorporating proof binding and the inclusion of Poseidon commitment of the underlying on-chain state (as described in Section 5.2.5), making cross-token replay attacks mathematically impossible rather than merely discouraged through application-layer checks. This is a known limitation of the prototype, and is addressed in more detail in Section 6.5.

6.5 Limitations

6.5.1 IPFS Integration via Pinata

The prototype utilizes Pinata's pinning service to store all credential metadata off-chain on IPFS (Pinata, 2023). In the prototype, when a credential is created, the front-end will upload the metadata JSON file to the Pinata API, and receive a content-addressed CID in response. The CID will then be stored on-chain in the metadataHash of the credential struct. This means that the credential metadata can be reliably retrieved by any party that has a CID and has been granted the appropriate access to retrieve it via the open IPFS gateway.

The only remaining service-dependent limitation is that of Pinata as a centralized third-party pinning service. If Pinata were to become unavailable to the user for any reason (e.g., the pinning subscription lapses, the service is suspended), the potential to access the pinned metadata over time could diminish due to an increasing number of nodes that hold a cached copy of the metadata dropping their cache. To mitigate this risk in a production environment, a self-hosted IPFS node could serve as a backup pinner to Pinata, thereby ensuring that metadata is not reliant solely upon one third-party provider for its availability.

6.5.2 Trusted Setup Trust Assumptions

The prototype ZKP circuits rely on the Hermez Network Powers of Tau Ceremony, specifically a phase 1 circuit-specific SRS (Hermez Network, 2020) and a single-creator phase 2 contribution. There are two trust assumptions associated with these circuits.

For phase 1, the security of the Hermez ceremony (55 contributors) is considered practically sound. There is a low probability that all contributors would collude together to retain the toxic waste.

For phase 2, the circuit-specific contribution was made by the project deployer alone. A single dishonest phase 2 contributor who retains their toxic waste can forge proofs for any statement without detection. In a production deployment, phase 2 should be a multi-party computation ceremony with independent participants and verifiable randomness, for example through a verifiable random beacon hash combined with the snarkjs zkey beacon command.

This limitation occurs due to the Groth16 proof system's trusted setup requirements and applies to all Groth16-based systems such as Polygon ID circuits (Polygon, 2022).

6.5.3 Browser-Only Proof Generation Scalability

Proof generation runs entirely in the browser in the prototype. For the small circuits deployed (37 KB WASM, 21 KB .zkey for inequality circuits), this is manageable on modern desktop hardware. Scalability concerns arise in three scenarios.

- Mobile devices: Low-end smartphones with limited memory and single-threaded JavaScript execution may fail to complete proof generation for the inequality circuits within acceptable time limits, as shown in Table 6.8.
- Extended circuits: If the circuits are extended with Poseidon commitments and 50-element score arrays as described in Section 5.2.5, the .zkey would grow significantly, potentially to several megabytes, making in-browser generation slower and the initial download cost higher.
- Concurrent users: In-browser proof generation places computation on the user's device. This is acceptable for a prototype but could limit the experience for a consumer-facing product with users on lower-end hardware.

Server-side proof generation, described in Section 3.7.2, resolves all three concerns for circuits whose inputs are already publicly available on-chain, and is the recommended evolution path for a production deployment.

6.5.4 Single Permissioned Network

The system is deployed on a private QBFT Besu consortium network with a single RPC endpoint. Three limitations follow from this.

- Single point of failure: If the Besu node at rpc.dimikog.org goes offline, the entire application becomes non-functional. A production deployment would require a redundant node cluster.

- Centralised network control: The QBFT validator set is controlled by a small group of participants. A truly decentralised deployment would use a public chain such as Ethereum mainnet or an L2, or a multi-organisation consortium.
- No cross-chain portability: Credentials issued on Chain ID 424242 are not portable to other chains. Cross-chain identity would require bridge contracts or a chain-agnostic DID resolution layer.

6.6 Answers to the Research Questions

6.6.1 RQ1: Privacy and Verifiability

Research Question 1: How can a decentralised identity platform based on soulbound tokens protect user privacy while maintaining the cryptographic verifiability of professional credentials?

SoulBound Identity provides an answer to this question through three architectural components. First, non-transferable identity anchoring using ERC-5484 (Cai, 2022) ties each soulbound credential explicitly to an unalterable wallet address, providing a cryptographically secure base for all soulbound credentials. As such, no soulbound credential can be transferred or delegated. Second, layered access control (bitmask per credential type in CredentialsHub, per social section in SocialHub, plus the request/approve/deny lifecycle in SoulboundIdentity) ensures that credential data is private by default and accessible only to parties the token owner explicitly approves. Finally, use of zero-knowledge proofs through usage of five Groth16 circuits (Groth, 2016), allows verification of certain specified threshold and category claim knowledge by a verifier without revealing any information concerning the actual source credential data. Consequently, this combination of the three components works well together and creates privacy at both levels of disclosure granularity. The combination is coherent: a verifier who needs only to confirm 'GPA above 3.0' uses a ZKP and receives a cryptographic proof without any credential access. A verifier who needs the full credential record requests access and, if approved, receives the IPFS-linked data. Privacy is preserved at both levels of disclosure granularity.

6.6.2 RQ2: ZKP Efficiency and User Transparency

Research Question 2: Can zero-knowledge proof systems provide efficient, user-transparent credential verification at scale on an EVM-compatible blockchain?

The findings from the evaluation demonstrate that both efficiencies of design and usability for a large number of users are met with positive confirmation.

Efficiency: The combined download size of all five proof circuits is only 249kb. This makes them smaller than most web page assets, and thus can be fully cached. The proof generation time for the equality proof circuits on modern desktop hardware was consistently <2 seconds, while that for the inequality proof circuits was consistently <5 seconds. On-chain verification takes approximately 280,000 gas units per proof to run, and this amount is fixed and non-scalable

(i.e. it is not dependent on the number of users using them). The Groth16 proof object is always 192 bytes in size, regardless of the complexity of the proof circuit being licensed, and as a result provides a minimal amount of calldata cost.

Usability: The ZKP Infrastructure is fully opaque to the end user. The front-end status display will give users some idea of what cryptographic activity has taken place via the status transitions of 'Generating, Verifying, Success' however that is the only means by which the end user will see that any such activity took place. The average user should not have access to circuit files, proof objects, elements of the field or Trusted Setup concepts. The Pre-Condition Check method described in Section 5.5.3 ensures that all circuit constraint failures generate plain-language error messages for the end users.

Limitation: It should be recognized that proof generation on low-end mobile devices via the browser can result in delays and/or non-responses that are longer than expectable for everyday users therefore if designed for broad market acceptance, server-side proof generation of the more complex proof-circuit types will likely be required.

6.6.3 RQ3: On-Chain Transparency vs. Selective Disclosure

Research Question 3: What is the appropriate design balance between on-chain transparency and selective disclosure in a professional identity system, and what access control mechanisms can mediate this balance?

The evaluation confirms that a three-tier disclosure model is both technically feasible and appropriate for professional credentials.

Tier 1 - Always public (on-chain, no access required): Structured fields needed for ZKP circuit inputs (GPA scaled by 100, category enums, work experience durations) and aggregate social data (review counts, running totals). These fields are public precisely because they serve as inputs to ZKP proofs that verifiers will request: making them public does not disclose more than the holder voluntarily proves through those proofs.

Tier 2 - Access-controlled (off-chain IPFS, requires approved grant): Rich metadata including institution names, credential titles, descriptions, and project details. These are accessible only to parties with an Approved status in SoulboundIdentity and the correct bitmask bits in CredentialsHub or SocialHub. The bitmask mechanism provides appropriate granularity: a recruiter interested only in degrees and certifications need not be granted access to identity proofs or skill endorsements.

Tier 3 - ZKP-only (cryptographic proof, no data disclosed): High-privacy threshold claims such as score above T, GPA above T, experience above T years, or category membership. These require no access grant: a verifier receives only a cryptographic proof and the claimed threshold. No underlying data is disclosed at any point.

The access control mechanisms that mediate this balance include: the AccessRequest state machine (for Tier 2 disclosure gating), the bitmask permission scheme (per-Type Tier 2 granularity) and the Groth16 verifier contracts (for Tier 3 mathematical guarantees). All of which provide a cohesive and composable enactment of an access model, wherein the Credential Owner maintains authority over how each tier level is disclosed.

Chapter 7 - Conclusion and Future Work

This chapter concludes the overall thesis. Section 7.1 summarises the contributions of the research. Section 7.2 provides a summary of the major research results based on the application and evaluation of the methods used. Section 7.3 returns to the limitations mentioned in Chapter 6 with a focus on being ready for production. Section 7.4 presents clear directions for future work in this area.

7.1 Summary of Contributions

This thesis set out to address a clearly defined research gap: no existing system combines soulbound non-transferable identity tokens, zero-knowledge proof selective disclosure, and granular per-credential-type access control in a single unified platform. SoulBound Identity serves as an initial prototype of a working proof of concept by providing these three characteristics within one solution operating on a permissioned EVM-compatible blockchain.

The primary contributions are as follows.

1. A fully deployed prototype. SoulBound Identity is not a theoretical proposal. Three Solidity smart contracts (SoulboundIdentity, CredentialsHub, and SocialHub) are deployed and operational on a QBFT Hyperledger Besu network (Chain ID 424242, Hyperledger, 2022). A React 18 single-page application provides a complete user interface covering identity creation, credential management, access control, social reputation, and ZKP-based verification. All end-to-end user journeys were executed and verified as part of the evaluation in Chapter 6.
2. A five-circuit ZKP subsystem. Five Circom 2 arithmetic circuits (reputation_threshold, gpa_threshold, work_experience_threshold, degree_category, and cert_domain) are compiled, have undergone trusted setup, and have corresponding Groth16 verifier contracts deployed on-chain (Groth, 2016; iden3, 2022). All five circuits were verified end-to-end: proofs generated in-browser, submitted to on-chain verifiers, and confirmed valid. The resulting overall 249KB total artifact size of all five circuits combined shows how practical ZKP verification is possible and results in significantly lower overhead than previous evaluation articles on browser-based ZKP verification had claimed.
3. A bitmask access control architecture. The CredentialsHub and SocialHub contracts implement a uint8 bitmask scheme that encodes per-credential-type and per-social-section access grants in a single storage slot. This design achieves O(1) permission reads, reduces grant-update gas cost by approximately 80 percent compared to a per-

grant mapping, and supports batch access operations that further amortise transaction costs. The SoulboundIdentity Access Request State Machine provides a full Access Governance Lifecycle by providing rate limiting, per-token cooldowns, time-bound approval periods, and allowing for batch approve/deny operations.

4. A user-transparent ZKP interface. The frontend demonstrates that ZKP generation can be made entirely invisible to end users. The flow of generating a proof moves through three status states (generating, verifying, success) without any indication of those states being sent via a plain text message to users, thus users never know about circuit files, field elements, etc. Pre-condition checks prevent failed proof attempts from being initiated, ensuring that the only user-visible failure mode is a clear, actionable message.
5. An empirical gas cost baseline. The evaluation in Chapter 6 provides a measured cost table for all major operations on an EVM-compatible chain, including the previously underspecified cost of Groth16 on-chain verification (Reitwiessner, 2017; Buterin & Reitwiessner, 2017). This baseline serves as a reference for future system designers evaluating the economic viability of ZKP-based credential verification on EVM chains.

7.2 Key Findings

Three findings emerge from the implementation and evaluation that have implications beyond this specific system.

Finding 1: ZKPs can be hidden from end users when working with practical sizes of circuits, therefore the view that ZKP systems have too great of a complexity for non-technical users is not a property of the technology, but a result of poor abstraction. At the circuit scales demonstrated in this thesis (37 KB WASM, 21 KB proving key for inequality circuits; 35 KB WASM, 3.5 KB proving key for equality circuits), in-browser proof generation is fast enough that users experience only a brief loading state on modern hardware, indistinguishable from any other asynchronous operation. The critical design principle is to abstract every ZKP concept behind the single question the user actually cares about: should this specific claim be shared? Nothing else needs to be communicated.

Finding 2: Groth16 on-chain verification is economically viable on EVM for high-value attestations. The approximately 280,000 gas cost of verifyProof is substantial relative to a simple token transfer (~21,000 gas) but modest relative to the value of the attestation it provides. Traditional pre-employment background checks are estimated to cost between USD 30 and USD 100 per candidate and take several days to complete (Verified First, 2023). At 20 Gwei and an ETH price of USD 3,000, a verified on-chain ZKP attestation costs approximately USD 0.017, several orders of magnitude less than its institutional equivalent. The EVM precompile support for BN128 (Reitwiessner, 2017; Buterin & Reitwiessner, 2017) is the enabling technical factor; without native pairing support, Groth16 verification would be economically impractical on-chain.

Finding 3: The IPFS off-chain split represents the correct abstraction boundary for credential storage. Storing rich credential metadata entirely on-chain is not economically viable. The cost of storing each additional 32-byte storage slot, under EIP-2200 (Volkov, 2019), is 20,000 gas. Thus, when storing even modest amounts of metadata using on-chain storage becomes prohibitively expensive as you scale. Storing everything off-chain loses the cryptographic verifiability property. The correct boundary stores on-chain exactly the fields that serve a cryptographic purpose (the structured inputs to ZKP circuits and the access-gated IPFS commitment hash) and stores everything else off-chain via a content-addressed storage system. This design is composable: the on-chain fields support ZKP proofs; the IPFS data supports full-disclosure access grants; and both levels of disclosure are independently controlled by the token owner.

7.3 Limitations Revisited

Chapter 6 documented the prototype's limitations. Viewed through the lens of production readiness, they fall into three categories.

Engineering limitations (addressable without research): The single-node network deployment (Section 6.5.4) is straightforward to resolve: network resilience requires a multi-node Besu cluster or migration to a public L2. IPFS metadata storage is already implemented via Pinata (Section 6.5.1), with the residual limitation being service dependency rather than missing functionality; a self-hosted redundant pinner would mitigate this. These are deployment engineering tasks, not research problems.

Design limitations (addressable with additional implementation): The absence of tokenId and data commitment binding in the prototype circuits (Section 6.4.5) means that ZKP replay protection is implemented in the application layer and not within the circuit itself. To provide ZKP replay proof functionality, Poseidon commitments (Grassi et al., 2019) will be added to the circuits along with adding tokenId to the circuit instance, and then, the addition of circuit constraints will allow the creation of a .zkey that is larger than those currently being used, which is acceptable in terms of performance for generation of .zkey on the user's device.

Architectural limitations (requiring design decisions): Browser-only proof generation (Section 6.5.3) is a deliberate prototype choice that trades infrastructure simplicity for scalability. The single-party phase 2 trusted setup (Section 6.5.2) trades ceremony overhead for time. Both require an architectural decision (server-side proof generation infrastructure, and a multi-party phase 2 ceremony) rather than code changes alone.

However, none of these variables make the core thesis findings invalid. In fact, they will provide clear guidance on how we move from prototype to production.

7.4 Future Work

7.4.1 Server-Side Proof Generation

The most significant improvement that can be achieved in the short term is moving proof generation from the browser to the Node.js Based Server, which is described in Section 3.7.2. Since all credential data used as ZKP circuit inputs is publicly available on-chain, server-side generation carries no privacy cost: the server reads the same on-chain data that the browser reads, generates the proof using snarkjs (iden3, 2023) in a native Node.js environment, and returns the proof to the frontend for on-chain submission.

The benefits are substantial: proof generation time will be reduced from multiple seconds to less than one second; the .wasm and .zkey files would be cached on the server and never downloaded by users; proof generation would work reliably on all mobile devices; and circuit updates would require only a server-side deployment rather than a frontend release. The API surface is minimal: a single POST /prove endpoint that takes the circuit name and public inputs and returns the proof object and public signals.

7.4.2 Production-Grade ZKP Circuits

The prototype circuits are basic proof-of-concept prototypes. A production ZKP sub-system will enhance each prototype with an additional 4 additions.

- Poseidon commitments (Grassi et al., 2021) binding the proof to a specific snapshot of on-chain data, preventing proof replay across different data states.
- TokenId binding as a public input, preventing cross-token proof reuse at the circuit level rather than relying on application-layer mitigations.
- Range checks via bit decomposition on numeric inputs, ensuring private inputs are within the valid domain and preventing trivially satisfying out-of-range witnesses.
- Merkle proof membership for the equality circuits, replacing the simple equality check with a proof that the credential's category is committed in a Merkle tree whose root is stored on-chain, enabling revocation without requiring a new proof.

These extensions will likely lead to increased counts of circuit constraints, which will increase .zkey file size by several megabytes. Combined with server-side generation (Section 7.4.1), this increased complexity remains transparent to end users.

7.4.3 Multi-Party Trusted Setup Ceremony

The single-contributor phase 2 trusted setup is the weakest security assumption in the current system. A production deployment should conduct a publicly verifiable multi-party computation phase 2 ceremony for each circuit, using a verifiable random beacon (such as an Ethereum block hash at a committed future block number) as the final contribution. The snarkjs toolchain supports this via the snarkjs zkey beacon command. With the participation of at least three to five independent contributors, the probability that a colluding majority would produce a compromised proving key is greatly diminished.

7.4.4 Pinata Redundancy and Encrypted IPFS Storage

Currently, IPFS metadata storage is managed through the use of Pinata (Pinata, 2023). The only major improvement needed is the addition of a backup pinner, via a redundant self-hosted IPFS node, to eliminate any reliance on any one third-party provider for the availability of IPFS metadata. This is simply a deployment configuration and will not require any changes to the existing smart contract or front-end code.

A further enhancement would be encrypted IPFS storage: encrypting credential metadata with a symmetric key held by the token owner before pinning, and sharing the decryption key alongside the CID with approved requesters. This would add a second layer of privacy protection, such that even a party who discovers the CID through on-chain observation would be unable to read the metadata without the decryption key. The encryption and decryption could be handled transparently by the frontend, consistent with the zero-knowledge transparency principle applied throughout the system.

7.4.5 Cross-Chain Identity Portability

The current deployment is tied to a single consortium network. Professional identity is inherently cross-organisational and cross-jurisdictional; a credentialing system that cannot cross chain boundaries has limited real-world applicability. Two paths towards portability exist.

The first is a DID-based resolution: anchoring the SoulBound Identity token as a DID document (W3C, 2022), for example did:ethr:0x424242:<tokenId>, and publishing it to a universal DID registry. Any resolver supporting the did:ethr method could then resolve the identity independently of the specific Besu network.

The second is cross-chain bridge integration: deploying bridge contracts that lock an SBT on the source chain and mint a read-only shadow on a destination chain. Credential ZKP proofs generated on the source chain could be verified on any chain that supports BN128 pairing precompiles (Reitwiessner, 2017; Buterin & Reitwiessner, 2017), which includes Ethereum mainnet and most major EVM-compatible L2 networks such as Polygon, Arbitrum, Optimism, and Base.

7.4.6 Towards ERC-5484 Extensions

The ERC-5484 standard as currently defined (Cai, 2022) specifies non-transferability and burn authorisation but leaves credential semantics, access control, and ZKP integration entirely to implementors. The SoulBound Identity implementation demonstrates a single implementation for one of the design spaces offered by the ERC-5484 standard. The patterns developed here, including bitmask credential type access, time-bounded access grants, and ZKP threshold proofs as a selective disclosure layer, could inform a proposed ERC extension that standardises these patterns at the interface level and enables interoperability between different SBT-based identity systems.

Contributing such an extension to the Ethereum Improvement Proposal process would represent a meaningful academic-to-standard pipeline outcome of this research.

7.5 Closing Remarks

The SoulBound Identity system ultimately demonstrates that the tension between the security of confidential credential data stored off-chain and the security afforded by on-chain storage of credential data is a practical (engineering) problem not a fundamental barrier. A principled engineering solution exists to create a coherent system of managing both the immutable storage of some data on-chain through cryptographic mechanisms and to protect the sensitive nature of other data through access controlled off-chain storage (e.g., through the use of ZKPs) which can be verified via access-controlled on-chain storage without revealing anything but the requisite information needed for the verification.

The SoulBound Identity system also supports that this engineering solution can be effectively implemented today: there are no gas costs that would be considered unreasonable for implementation; the ZKP toolsets have matured to an extent that they can be operated from any modern web browser; the design patterns and attributes for building Smart Contracts are composable and audit-able. As such, the pathway from the prototype described in this thesis to an operational solution consists of documented and verifiable technical engineering issues versus a lack of resolved fundamental research issues.

The research questions made in Chapter 1 have been answered affirmatively. Privacy and verifiability are not in conflict; they are complementary properties achievable within a single coherent architecture. The appropriate balance between on-chain transparency and selective disclosure is not a fixed point but a three-tier design space, always public, access-controlled, and ZKP-only, that the token owner navigates according to the sensitivity of each disclosure. And the ZKPs required to implement this balance are, at the scales relevant to professional credential verification, efficient enough to be invisible.